Quantum Complexity Theory

John Watrous
Institute for Quantum Computing
University of Waterloo

Part II

Quantum polynomial time

Chapter 4

The quantum circuit model

With a basic foundation in classical complexity theory established by the three previous chapters, we are now prepared to begin a proper discussion of quantum complexity theory, starting with a chapter on the quantum circuit model. A familiarity with quantum circuits, and quantum information in general, is assumed — so it is not the aim of the chapter to explain the rudiments of quantum circuits, but rather to formalize the model in support of subsequent chapters.

In classical complexity theory, the Turing machine model has central importance, essentially serving as a basis for the entire theory. Boolean circuits are also important and of broad interest in their own right, but there is no sense in which they challenge the foundational role of the Turing machine. It is therefore only natural that, in the early days of quantum computing, researchers turned to quantum variants of the Turing machine model as a basis for quantum complexity theory.

As it turns out, working with quantum Turing machines is comparatively difficult, due in no small part to the tediousness of designing and describing them under constraints that guarantee global unitary evolution. For this reason, motivated by their relative ease of use, quantum circuits became the default model for quantum computing by the mid-1990s.

In 1993, Yao proved that polynomial-time quantum Turing machines are equivalent in power to polynomial-time uniform families of quantum circuits. The proof does have some resemblance to the proof of the analogous classical fact, which we discussed in the previous chapter, but the simulation of quantum Turing machines by quantum circuits is much more delicate. In particular, the simulation must effectively break the global unitary evolution of a quantum Turing machine into local

unitary components, which is nontrivial. We will not cover this proof or discuss quantum Turing machines at a technical level in this course, but instead directly adopt quantum circuits as a foundational model for quantum computing.

It should be stressed, however, that this does not mean that we will abandon Turing machines altogether. Indeed, there is a sense in which the fundamental importance of classical Turing machines in complexity theory continues throughout quantum complexity theory, beginning with the fact that our focus will not merely be on quantum circuits, but on uniform families of quantum circuits generated by classical Turing machines. Without classical Turing machines or an equivalent model we would not be able to define BQP, QMA, or other quantum complexity classes of interest.

4.1 General quantum circuits

Model definition

We will begin by considering how quantum circuits can be formally defined as mathematical objects. The most typical definitions are phrased in intuitive, human-readable terms — such as *an acyclic network of quantum gates connected by wires*. While such definitions are generally adequate and unambiguous, they do leave a gap between the concept of a quantum circuit and foundational mathematical objects such as sets, functions, and so on. Other formal definitions, including ones based on category theory, tensor networks, and programming languages, leave no such gaps, but can be difficult to grasp for those unfamiliar with the notation and terminology used within these specific topics.

With this in mind, here is a formal definition for quantum circuits in the tradition of theoretical computer science.

Definition 4.1. A *quantum circuit* Q is defined by the following objects:

1. A finite set G indexing the individual gates of Q.
2. Finite sets S and T, labelling the input and output qubits of Q, along with finite sets S_g and T_g for each $g \in G$, labelling the input and output qubits of the gate indexed by g. The sets S, T, S_g, and T_g (for all $g \in G$) must all be disjoint.
3. A one-to-one and onto function w of the form

$$w : S \cup \bigcup_{g \in G} T_g \to T \cup \bigcup_{g \in G} S_g \tag{4.1}$$

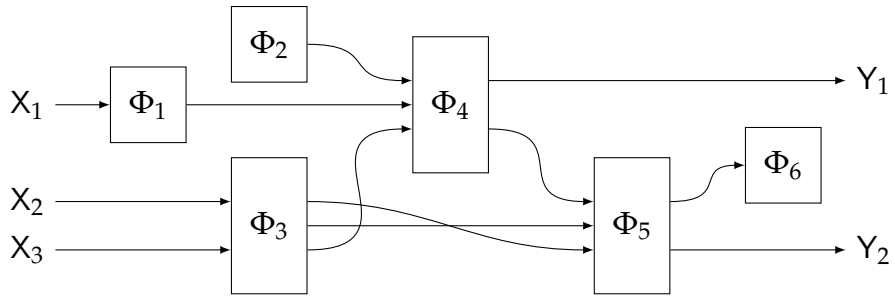

Figure 4.1: A general quantum circuit with three input qubits X_1, X_2, X_3, two output qubits Y_1, Y_2, and gates having associated channels $\Phi_1, \ldots, \Phi_6$. With respect to Definition 4.1, the set of input qubits is $S = \{X_1, X_2, X_3\}$, the set of output qubits is $T = \{Y_1, Y_2\}$, and the set of gates is $G = \{1, \ldots, 6\}$. The input and output qubits for each gate are not explicitly named.

that describes the wires of Q. Specifically, $w(j) = k$ indicates a wire identifying qubit j (an input qubit of Q or an output qubit of one of its gates) with qubit k (an output qubit of Q or an input qubit of one of its gates).

4. A quantum channel Φ_g for each $g \in G$, transforming a collection of qubits indexed by S_g into a collection of qubits indexed by T_g.

A circuit Q is *acyclic* if the directed graph with vertex set G and edge set

$$E = \big\{(g,h) \,:\, w(T_g) \cap S_h \neq \varnothing\big\} \tag{4.2}$$

is acyclic, and its *size* is defined to be its number of gates: $\mathrm{size}(Q) = |G|$.

Hereafter, whenever we use the term *quantum circuit*, we specifically mean *acyclic quantum circuit*. Quantum circuits that are not acyclic do find obscure uses, but will not be considered further in this course.

Note that Definition 4.1 implies that every quantum circuit must satisfy

$$|S| + \sum_{g \in G} |T_g| = |T| + \sum_{g \in G} |S_g|, \tag{4.3}$$

for otherwise the function w cannot be one-to-one and onto. Intuitively speaking, every qubit must be accounted for: each of the input qubits and the output qubits of the individual gates must be identified with a unique output qubit of the circuit or input qubit of one of the gates, and there can be no "dangling" qubit wires in the circuit.

Figure 4.1 shows an example of a quantum circuit with three input qubits and two output qubits containing gates of varying numbers of inputs and outputs. The gate Φ_2, for instance, has 0 input qubits and 1 output qubit, so this gate effectively represents the initialization of a qubit in some state (determined by the channel Φ_2, which is equivalently a state because it has no inputs). The gate Φ_6, on the other hand, has one input qubit and no output qubits, so there is only one possibility for the corresponding channel: it must be the partial trace over one qubit. The other channels have both input and output qubits and could be arbitrary, provided that the number of input and output qubits agrees with the diagram.

The channel computed by a circuit

Every quantum circuit computes a channel transforming its input qubits into its output qubits. This channel is obtained by composing the channels corresponding to the individual gates of Q, using the wiring w to route qubits between the gates.

To make this idea more precise, it is helpful to introduce the notion of a *topological ordering* of a directed acyclic graph. A topological ordering of a directed acyclic graph is a total ordering of its vertices such that, for every directed edge (u, v), the vertex u comes before v in the ordering. Every directed acyclic graph admits a topological ordering, and indeed this property characterizes directed acyclic graphs. Such an ordering can be computed efficiently.

For an acyclic quantum circuit Q, the directed graph with vertex set G and edge set E, as in Definition 4.1, is acyclic. In words, the existence of a directed edge (g, h) in this graph indicates that a wire goes from an output qubit of the gate indexed by g to an input qubit of the gate indexed by h. Given a topological ordering $g_1 < g_2 < \cdots < g_k$ of the gates of G, the channel computed by the quantum circuit Q is obtained by applying the gate channels $\Phi_{g_1}, \Phi_{g_2}, \ldots, \Phi_{g_k}$ in order, using the wiring w to identify which qubits the channels act upon.

The resulting channel maps the qubits indexed by S to the qubits indexed by T. We will use the same letter that names a given circuit to refer to its associated channel — so if Q is a quantum circuit and ρ is a density matrix describing a state of its input qubits, then $Q(\rho)$ is the density matrix for its output qubits. Overloading the meaning of Q, or any other name for a circuit, will not cause any confusion, as it will always be clear from context which object is being referred to.

Note that the channel obtained from a circuit in this way does not depend on which particular topological ordering is used. One way to argue this is to make use of the fact that any two topological orderings of a directed acyclic graph are related by a sequence of transpositions of adjacent vertices that are not connected

by an edge. In the circuit context, such a transposition amounts to swapping the order in which two channels act on *disjoint* sets of qubits, which does not change the composed channel. Iterating this transposition argument reveals that the final channel is independent of the specific topological ordering.

Unitary quantum circuits

We commonly restrict our attention to quantum circuits for which every gate is unitary. Such circuits are naturally called *unitary quantum circuits*.

Definition 4.2. A *unitary quantum circuit* is a quantum circuit in which the channel Φ_g corresponding to the gate label g is unitary, for every $g \in G$.

Unitary gates output the same number of qubits as they input, so a unitary quantum circuit must satisfy $|S_g| = |T_g|$ for every gate $g \in G$. It follows that $|S| = |T|$; the number of input and output qubits of a unitary quantum circuit must agree.

One nice property of unitary quantum circuits is that they can be run backwards. As the following definition clarifies, we will refer to the resulting circuit as the *reverse* of the original circuit.

Definition 4.3. Let Q be a unitary quantum circuit, with S, T, G, S_g, T_g, and w all as in Definition 4.1. The reverse of Q is the quantum circuit R defined by swapping the sets S and T, swapping the sets S_g and T_g for each $g \in G$, replacing w by its inverse, and replacing each unitary channel $\Phi_g(\rho) = U_g \rho U_g^*$ with its inverse channel $\Phi_g^*(\rho) = U_g^* \rho U_g$.

The quantum circuit R obtained in this way computes the inverse of the channel computed by Q. That is, if $Q(\rho) = U\rho U^*$, then $R(\rho) = U^*\rho U$.

Hereafter, we will typically use names such as U, V, and W for unitary quantum circuits to help to distinguish them from general quantum circuits. It will also sometimes be convenient to associate the name of a unitary quantum circuit with the unitary matrix that it describes, as opposed to the channel it describes, but this will always be made clear in the context in which it is done.

Unitary dilations of quantum circuits

Every quantum channel Φ from n qubits to m qubits has a *unitary dilation*, or equivalently a *Stinespring representation*. This means that there are natural numbers r

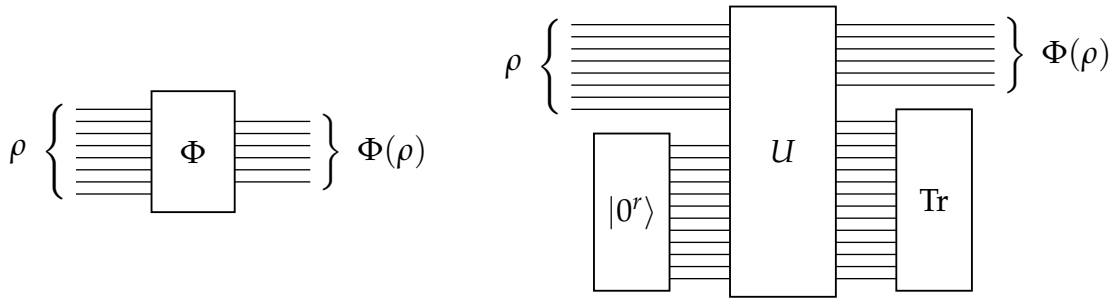

Figure 4.2: On the left, a quantum channel Φ taking a state ρ on n qubits to $\Phi(\rho)$ on m qubits. On the right is a Stinespring representation of this channel, in which r ancillary qubits are introduced, a unitary operation U is applied to the qubits, and finally s garbage qubits are traced out, leaving m output qubits in the state $\Phi(\rho)$.

and s satisfying $n + r = m + s$, along with a unitary operation U on $n + r$ (equivalently $m + s$) qubits, such that

$$\Phi(\rho) = \left(\mathbb{I}^{\otimes m} \otimes \mathrm{Tr}^{\otimes s}\right)\left(U\left(\rho \otimes |0^r\rangle\langle 0^r|\right)U^*\right) \tag{4.4}$$

for every n-qubit state ρ, with the understanding that $\mathbb{I}$ is the identity channel and Tr is the partial trace, both on one qubit. In words, if we input n qubits in an arbitrary state, initialize r qubits each to the $|0\rangle$ state, apply the operation U to all of the qubits, and finally discard (i.e., trace out) the last s of the resulting qubits, the net effect is to apply the channel Φ. The r initialized qubits will be called *ancillary qubits* and the s qubits that are traced out will be called *garbage qubits* when it is necessary to refer to them by name. There is always such a representation, provided $s \geq n + m$, or equivalently $r \geq 2m$. Figure 4.2 illustrates this relationship for $n = 8$, $m = 6$, $r = 12$, and $s = 14$.

When we refer to a unitary dilation of a *quantum circuit* Q, we mean a unitary quantum circuit that implements a unitary dilation of whatever channel the original quantum circuit performed. We can always obtain a unitary quantum circuit like this by simply dilating each of the individual gates in the circuit and treating all of the additional input and output qubits required by these dilations as inputs and outputs of the unitary circuit.

Specifically, for each gate $g \in G$, if Φ_g is a channel from n_g qubits to m_g qubits, then it can be dilated to a unitary operation U_g on $n_g + r_g = m_g + s_g$ qubits, where $r_g \geq 2m_g$ (or equivalently $s_g \geq n_g + m_g$). Doing this for every gate creates a final unitary circuit that, in addition to the $n = |S|$ input qubits of Q, takes an additional $r = \sum_{g \in G} r_g$ input qubits (which one might hope are initialized, though the unitary circuit itself has no way to enforce this). As a result, this circuit produces an

additional $s = \sum_{g \in G} s_g$ qubits beyond the $m = |T|$ qubits output by Q. The wiring map w for Q can be extended to obtain one for this unitary circuit in a straightforward way, where all of the additional qubits needed for the gate dilations are wired directly to inputs or outputs of the unitary circuit. The ordering of the new input and output qubits can be arbitrary, though it is natural to base the ordering on a topological ordering of the gates of the circuit.

4.2 A standard gate set

Definition 4.1 allows arbitrary channels to describe the actions of gates. To obtain a computational model that measures computational cost in a meaningful way, in terms of circuit size, we must therefore limit the choices for gates. (Otherwise, arbitrarily complex quantum computations could be viewed as channels implemented by unit-cost gates.) One way to proceed is to formulate specific conditions on quantum channels through which they may be deemed to correspond to single gates. More commonly, however, we simply pick a so-called *standard gate set*.

Hadamard, S, and Toffoli gates

The standard gate set we will choose for this course includes these three unitary gates: Hadamard, S, and Toffoli gates. Hadamard and S gates have the following descriptions as unitary matrices:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad \text{and} \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}. \tag{4.5}$$

Toffoli gates have the action

$$\text{TOF} : |a\rangle|b\rangle|c\rangle \mapsto |a\rangle|b\rangle|c \oplus ab\rangle \quad (\text{for all } a, b, c \in \{0,1\}) \tag{4.6}$$

on standard basis states, which corresponds to an 8×8 permutation matrix.

Two simple non-unitary gates

In addition to the unitary gates just described, it is also convenient to include these two non-unitary gates in our standard gate set:

- *Ancillary gates* take no input qubits and output one qubit initialized to the state $|0\rangle$. We will denote this gate symbolically by $|0\rangle$, because this is essentially what it is: the preparation of the state $|0\rangle$, viewed as a gate.

- *Erasure gates* take one input qubit and produce no output qubits. As channels they are described by the partial trace, and correspondingly we will denote them symbolically by Tr.

There is a sense in which the function or purpose of these two gates in quantum circuits is largely cosmetic. Rather than using ancillary gates to produce initialized qubits, one can instead assume that initialized qubits are included as inputs; and rather than using erasure gates to trace out qubits, one can simply disregard them after computations are performed, which is tantamount to tracing them out. On the other hand, the inclusion of these gates in a standard gate set provides a simple, natural, and meaningful way to include this information — namely, which input qubits to a unitary circuit are presumed to be initialized and which are presumed to be discarded after the circuit is run — directly in a particular circuit.

Note that the unitary dilation construction from Section 4.1 becomes essentially trivial for circuits composed of unitary gates, ancillary gates, and erasure gates. Specifically, nothing is done to the unitary gates while the ancillary and erasure gates are dilated to single-qubit identity gates — so they are simply removed and the corresponding input or output qubit is classified as one to be initialized or discarded accordingly.

Remarks on Hadamard, controlled-NOT, and T gates

The gate set above is not the most common choice of a gate set in quantum computing. More commonly, the unitary gates S and TOF are replaced by T and CNOT gates, which are described by unitary matrices as follows.

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & \frac{1+i}{\sqrt{2}} \end{pmatrix} \quad \text{and} \quad \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \tag{4.7}$$

For the CNOT operation specifically, this matrix representation describes the action $|a\rangle|b\rangle \mapsto |a\rangle|a \oplus b\rangle$, where the first qubit is the *control* and the second qubit is the *target*.

It is easy to show that Toffoli gates can be implemented using Hadamard, CNOT, and T gates; Figure 4.3 shows a circuit that does the job. In contrast, it is not possible to exactly implement T gates using H, S, and TOF gates, though these gates can (along with ancillary and erasure gates) approximate T gates to any desired degree of accuracy.

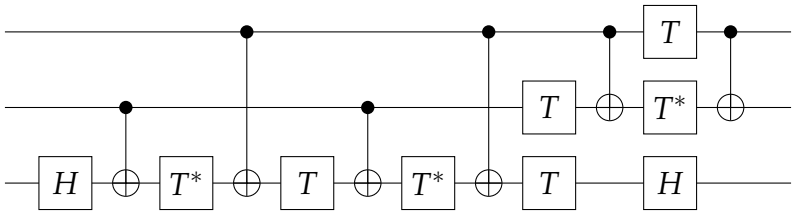

Figure 4.3: An implementation of a Toffoli gate as a product of Hadamard, CNOT, T, and T^* gates. The top two qubits are the control qubits and the bottom qubit is the target, so that the circuit computes $|a\rangle|b\rangle|c\rangle \mapsto |a\rangle|b\rangle|c \oplus ab\rangle$. It is the case that $T^* = T^7$, so each T^* gate can be replaced by seven T gates if T^* gates are deemed forbidden.

The reason we choose S and TOF over T and CNOT is a simple matter of "algebraic convenience." This will become more clear later on, particularly in Chapter 6. For now, the short explanation is that working with H, S, and TOF gates allows us to describe the actions of quantum circuits exactly through arithmetic on numbers whose real and imaginary parts are rational.

Example of an unreasonable gate selection

There are other choices one could make for a standard gate set, but some choices are not reasonable. Here is an example adapted from one due to Adleman, DeMarrais, and Huang.

Choose an undecidable subset $A \subseteq \mathbb{N}$ (e.g., $n \in A$ if and only if the n-th 1-DTM halts on the empty string, according to some enumeration of 1-DTMs), and define

$$\alpha = 2\pi \sum_{n \in A} 4^{-(n+1)}. \tag{4.8}$$

By performing phase estimation on the unitary operation

$$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\alpha} \end{pmatrix}, \tag{4.9}$$

one can (with sufficient time) compute whether a given $n \in \mathbb{N}$ is or is not in A with bounded error.

Hence, undecidable problems become solvable with this unreasonable gate. The problem with it is that $e^{i\alpha}$ is not computable, whereas any reasonable notion of a standard gate set must be restricted to gates whose entries are (at a bare minimum) computable. Indeed, the entries must be *efficiently* computable for any gate that is considered to be reasonable.

4.3 Universality

A set of quantum gates is *universal* if it can be used to approximate any quantum channel, from qubits to qubits, to any desired degree of accuracy with a circuit composed of gates from this set. It is important to clarify that no restrictions are placed on the size of circuits needed to approximate a given channel.

In this context, approximation is defined in terms of the diamond-norm distance:

$$\delta(\Phi, \Psi) = \tfrac{1}{2} \left\| \Phi - \Psi \right\|_{\diamond}. \tag{4.10}$$

In essence, this is the maximum trace distance between the outputs of two channels on a common state, including the possibility that the input qubits to the channels are entangled with a second system. In operational terms, this distance quantifies how well two channels can be distinguished through an input state preparation followed by a post-channel measurement.

We will occasionally write $\delta(\cdot, \cdot)$ with arguments that are not literally channels. In particular, when a unitary operation U appears as one of the arguments, it is to be understood that this is a shorthand for the channel $\rho \mapsto U\rho U^*$ it induces. For instance, $\delta(\Phi, U)$ means the diamond-norm distance between Φ and the channel $\rho \mapsto U\rho U^*$.

Universality is commonly defined only for *unitary* operations rather than general channels, but this distinction is superficial; if one can approximate a unitary dilation of a given channel, then that channel is approximated to the same degree of accuracy once the ancillary and garbage qubits are specified.

Statement of the universality theorem

The following theorem states the key fact concerning universality that will be used in this course.

Theorem 4.4 (Universality theorem)**.** *Let Φ be a channel from n qubits to m qubits. For every $\varepsilon > 0$, there exists a quantum circuit Q, composed of H, S,* TOF*, ancillary, and erasure gates, such that $\delta(\Phi, Q) < \varepsilon$. Moreover, for fixed n and m, the circuit Q can be chosen so that* $\text{size}(Q) = \text{poly}(\log(1/\varepsilon))$.

The theorem is also true as stated for H, T, and CNOT gates in place of H, S, and TOF gates — and indeed these unitary gates can be replaced by any universal set of unitary gates. The proof combines the Solovay–Kitaev theorem (stated below), which concerns unitary approximation on a *fixed* number of qubits, with standard

methods for reducing arbitrary unitary operations to one- and two-qubit unitary operations.

It should be noted that an exponential dependence on n and m of the size of a circuit Q approximating a given channel Φ cannot be avoided: by a measure-theoretic argument analogous to the classical counting argument for Boolean circuits, most channels require exponentially many gates to approximate, regardless of what finite gate set is chosen.

The Solovay–Kitaev theorem

One of the components needed to prove the universality theorem is the following theorem. A preliminary version of it was announced by Robert Solovay to an email list in 1995 (though he never released a paper) and Alexei Kitaev independently proved a more general version in 1997.

Theorem 4.5 (Solovay–Kitaev)**.** *Let K be a finite set of d-dimensional unitary channels that 1) is closed under inverses, and 2) generates a dense subgroup of the d-dimensional unitary channels. There is a constant c such that, for every d-dimensional unitary channel Φ and every $\varepsilon > 0$, there exists a product of $O(\log^c(1/\varepsilon))$ channels in K that approximates Φ to within ε in δ-distance. Moreover, such a product can be found in time polynomial in $\log(1/\varepsilon)$.*

The proof of this theorem is involved and will not be covered in this course. A proof can be found in the 2002 book *Classical and Quantum Computation* by Kitaev, Shen, and Vyalyi. The original analysis by Kitaev gave $c = 3 + \gamma$ for any $\gamma > 0$, which Kuperberg reduced to $c \approx 1.44$ in 2023. A lower bound of $c \geq 1$ is also known.

Regarding our standard gate set, we note that $\{H, S, \mathrm{TOF}\}$ is not literally closed under inverses, nor do all of the gates act on the same number of qubits, but the theorem can still be applied to this gate set. For the first point, we observe that $S^{-1} = S^3$. Alternatively, we can tacitly assume our gate set includes S^* along with S; it is typical to assume that unitary gates and their inverses are equivalent with respect to implementation cost. For the second point, H and S can be tensored with the identity operation on two qubits.

Reconciliation with Adleman–DeMarrais–Huang

The Solovay–Kitaev theorem suggests that the specific choice of gate set is secondary for polynomial-time complexity: any two (finite and inverse-closed) gate

sets could be exchanged for one another, provided the dense condition is true for both. On the other hand, the Adleman–DeMarrais–Huang example from the previous section seems to suggest the opposite: some gate sets are powerful enough to decide undecidable problems with bounded error probability.

Of course, both statements are correct. The apparent tension is resolved by clarifying the assumptions and implications of the theorems. The Solovay–Kitaev theorem is a statement about approximating a target unitary by products from a fixed gate set, but actually *finding* such a product efficiently requires the entries for both the gate set and the target unitary to be efficiently computable. The Adleman–DeMarrais–Huang example, in contrast, violates this requirement in an extreme way by effectively encoding uncomputable information into a single entry of a unitary gate.

Exact synthesis

Here is one final remark for the chapter, though it is mostly an aside from the viewpoint of this course. For the specific gate set $\{H, \text{CNOT}, T\}$, it is possible to exploit the algebraic structure of this set of gates to effectively do better than the Solovay–Kitaev theorem states.

Theorem 4.6. *A $2^n \times 2^n$ unitary matrix U can be implemented exactly (up to a global phase) by a quantum circuit composed of Hadamard,* CNOT*, T, ancillary, and erasure gates if and only if every entry of U lies in $\mathbb{Z}[\omega, \frac{1}{2}]$ for $\omega = e^{i\pi/4}$.*

The $n = 1$ case of this theorem, together with an efficient synthesis algorithm, was proved by Kliuchnikov, Maslov, and Mosca, requiring just Hadamard and T gates for the implementation. The multi-qubit case was proved shortly after by Giles and Selinger, whose construction requires the use of a small number of ancillary qubits.

The exact-synthesis characterization enables a stronger approximate synthesis, in the following sense: given a target single-qubit unitary that need not lie in $\mathbb{Z}[\omega, \frac{1}{2}]$, one can search for a nearby unitary that does, and then invoke the exact-synthesis algorithm on it. Ross and Selinger gave such a search that approximates any single-qubit unitary to accuracy ε using $O(\log(1/\varepsilon))$ gates — which is essentially optimal and an improvement over what the Solovay–Kitaev theorem alone provides.

Chapter 5

Bounded-error quantum polynomial time

We now turn to the definition of BQP, short for *bounded-error quantum polynomial time*. The complexity class BQP is central to quantum complexity theory, serving as an abstraction of the decision problems that can be solved efficiently using a quantum computer.

We will adopt a definition based on the quantum circuit model (as is standard), through which the notion of "quantum polynomial time" is formalized in terms of polynomial-time uniform families of quantum circuits. In addition to defining BQP, this chapter discusses error reduction for BQP and the degree to which this class is independent of a chosen gate set.

5.1 The definition of BQP

Encodings and polynomial-time uniformity

The first step toward defining BQP using the quantum circuit model is to extend the notion of polynomial-time uniformity, which we already discussed for Boolean circuits in Chapter 3, to quantum circuits.

To begin, we assume that a reasonable and efficient encoding scheme for quantum circuits over a fixed gate set has been selected. Rather than describing the

properties such a scheme must possess, it is easier to simply describe a suitable scheme in high-level terms.

An encoding of a quantum circuit Q separately encodes each of the finite sets involved in the description of Q, including the input qubits S, the output qubits T, the set G indexing the gates, and, for each gate $g \in G$, the input and output qubits S_g and T_g of that gate. In addition, the encoding specifies the wiring map w and, for each $g \in G$, the identity of the element of the standard gate set that Φ_g implements. Note that the channel descriptions themselves need not appear in the encoding, as they are assumed to be drawn from our fixed standard gate set.

A scheme of this general form can be selected that has the properties one expects: encodings have length polynomial in the total number of qubits and gates, distinct circuits have distinct encodings, and the structural properties of a circuit are extractable from its encoding by a polynomial-time DTM. Moreover, there are no unreasonably compact encodings, and in particular we may assume that the total numbers of qubits and gates in a quantum circuit are bounded by the length of the encoding of that circuit.

With such a scheme in hand, the notion of polynomial-time uniformity can be defined in an analogous way to uniform families of Boolean circuits. It will, however, be convenient for our purposes to generalize the definition as follows.

Definition 5.1. For a given alphabet Σ, a family

$$\{Q_x : x \in \Sigma^*\} \tag{5.1}$$

of quantum circuits is *polynomial-time uniform* if there exists a polynomial-time DTM that, on each input $x \in \Sigma^*$, outputs an encoding of the circuit Q_x. When such a family is indexed by the natural numbers, as in

$$\{Q_n : n \in \mathbb{N}\}, \tag{5.2}$$

it is to be assumed that each $n \in \mathbb{N}$ is encoded in unary notation as 0^n. That is, (5.2) is polynomial-time uniform if there exists a polynomial-time DTM that, on each input 0^n, outputs an encoding of the circuit Q_n.

The value in extending the notion of uniformity in this way is that it allows input strings to effectively be hard-coded, or "baked in," to quantum circuits. In practical terms, this allows us to avoid being forced to use qubits to store classical input strings, which turns into notational clutter. More philosophically, this general definition represents a practical abstraction of quantum computing, where a classical computer is given an input (e.g., an integer to factor or the description of

a Hamiltonian whose ground state energy is to be approximated), and it implements a quantum computation to solve the problem. There is no reason why the input must be presented to the quantum circuit as a standard basis state when the circuit itself may depend on the input string.

Focusing on the channels computed by a given polynomial-time uniform family $\{Q_x : x \in \Sigma^*\}$ of quantum circuits, we observe that no generality is lost in assuming that the size of each Q_x depends only on the length of each input string x. That is, we may assume that there exists a polynomial resource bound s such that

$$\text{size}(Q_x) = s(|x|) \tag{5.3}$$

for every $x \in \Sigma^*$. This follows from the fact that gates can easily be added to circuits without changing the channel they compute. For example, the addition of an ancillary gate, followed by an arbitrary number of S or H gates on the qubit produced, followed by an erasure gate on this qubit adds any desired number of gates (as long as the number is at least two) to a circuit without affecting the circuit's channel description.

The following proposition, which takes this simple idea one step further for unitary dilations of certain families, will be convenient in Chapter 6.

Proposition 5.2. *Let $\{Q_x : x \in \Sigma^*\}$ be a polynomial-time uniform family of quantum circuits, where each Q_x has $p = p(|x|)$ input qubits and $r = r(|x|)$ output qubits, for polynomial resource bounds p and r. There exist polynomial resource bounds q and s and a polynomial-time uniform family $\{U_x : x \in \Sigma^*\}$ of unitary quantum circuits such that each U_x is a unitary dilation of Q_x that uses exactly $q = q(|x|)$ ancillary qubits and has size exactly $s = s(|x|)$.*

Proof. For circuits over our standard gate set, the dilation described in Section 4.1 is trivial: the unitary gates are left alone, the ancillary and erasure gates are deleted, and the qubits associated with the ancillary gates and erasure gates are classified as ancillary and garbage qubits, respectively. Let V_x denote the unitary circuit that results from dilating Q_x in this way for each $x \in \Sigma^*$.

Next, let q be a polynomial resource bound such that $q(|x|)$ is both positive and at least the number of ancillary qubits of V_x. Such a polynomial resource bound exists by the assumption that $\{Q_x : x \in \Sigma^*\}$ is polynomial-time uniform. Let W_x be the unitary circuit obtained by adding additional "dummy" ancillary qubits to V_x so that W_x has exactly $q(|x|)$ ancillary qubits, for each $x \in \Sigma^*$. Note that each W_x has at least one ancillary qubit because $q(|x|)$ is positive.

Finally, let s be a polynomial resource bound such that $s(|x|)$ upper-bounds the size of W_x, which agrees with the number of unitary gates of Q_x. Again, such a polynomial resource bound exists by the assumption that $\{Q_x : x \in \Sigma^*\}$ is polynomial-time uniform. Let U_x be the unitary quantum circuit obtained by prepending a number of S gates to the first ancillary qubit of W_x so that $\text{size}(U_x) = s(|x|)$. Any S gates added in this way have no effect, as S gates act trivially on the $|0\rangle$ state. It is clear that $\{U_x : x \in \Sigma^*\}$ is polynomial-time uniform and satisfies the requirements of the proposition. □

Bounded-error quantum polynomial time

With the notion of polynomial-time uniformity for quantum circuits in hand, we are ready to define BQP. It is convenient to define a parameterized version of BQP with arbitrarily chosen acceptance threshold probabilities for yes and no inputs, with BQP itself corresponding to the standard bounded-error thresholds $2/3$ and $1/3$, respectively.

Definition 5.3. Let $A = (A_{\text{yes}}, A_{\text{no}})$ be a promise problem and let $\alpha, \beta : \mathbb{N} \to [0,1]$ be functions (or constants). Then $A \in \text{BQP}(\alpha, \beta)$ if there exists a polynomial-time uniform family $\{Q_x : x \in \Sigma^*\}$ of quantum circuits, where each Q_x has no input qubits and one output qubit, such that

$$x \in A_{\text{yes}} \implies \langle 1|\rho_x|1\rangle \geq \alpha(|x|), \tag{5.4}$$

$$x \in A_{\text{no}} \implies \langle 1|\rho_x|1\rangle \leq \beta(|x|), \tag{5.5}$$

where ρ_x denotes the single-qubit state output by Q_x. The complexity class BQP is then defined as $\text{BQP} = \text{BQP}(2/3, 1/3)$.

The expression $\langle 1|\rho_x|1\rangle$ in the definition is the probability that a standard basis measurement of the output qubit of Q_x results in 1, while $\langle 0|\rho_x|0\rangle$ is the probability of obtaining 0. When it is convenient to do so, we refer to the associated events as Q_x *accepting* or *rejecting*, respectively, so that

$$\Pr(Q_x \text{ accepts}) = \langle 1|\rho_x|1\rangle, \tag{5.6}$$

$$\Pr(Q_x \text{ rejects}) = \langle 0|\rho_x|0\rangle. \tag{5.7}$$

A standard example of a problem in BQP is *integer factorization*, which is in BQP as a result of Shor's algorithm. To be more precise, because BQP is a class of

decision problems (including languages and promise problems more generally), we are speaking here of integer factorization expressed as a *language*:

$$\text{FACTORING} = \{\langle m, k\rangle \,:\, \exists d \in \{2, \ldots, k\} \text{ such that } d \text{ divides } m\}. \tag{5.8}$$

This is a very special example for multiple reasons, including the fact that it is a language as opposed to a promise problem having a nontrivial promise. Many other commonly discussed problems in BQP are genuinely promise problems, such as this example of a BQP-complete problem (with respect to Karp reductions).

BOUNDED-ERROR QUANTUM CIRCUIT ACCEPTANCE

Input	The encoding of a quantum circuit Q over the standard gate set with no input qubits and one output qubit.
Yes	$\Pr(Q \text{ accepts}) \geq 2/3$.
No	$\Pr(Q \text{ accepts}) \leq 1/3$.

Simulating probabilistic computations

To conclude this section, let us prove that BPP $\subseteq$ BQP, which is straightforward. The key observations we require are 1) Boolean circuits can be directly simulated by quantum circuits, and 2) random bits can be generated by quantum circuits. In particular, Figure 5.1 shows how AND, OR, and FANOUT gates can be simulated using Toffoli, controlled-NOT, NOT, ancillary, and erasure gates; and how a uniform random bit can be produced using H, controlled-NOT, and erasure. Note that NOT and controlled-NOT gates are easily simulated by quantum circuits using the standard gate set as illustrated in Figure 5.2.

Proposition 5.4. BPP $\subseteq$ BQP.

Proof. Let $A = (A_{\text{yes}}, A_{\text{no}}) \in$ BPP, so that there exist a polynomial resource bound p and a language $B \in$ P as in Definition 2.12. That is, if $x \in A_{\text{yes}}$, then $\langle x, y\rangle \in B$ for most (i.e., at least a $2/3$ fraction of) strings $y \in \{0,1\}^p$; and if $x \in A_{\text{no}}$, then $\langle x, y\rangle \notin B$ for most $y \in \{0,1\}^p$.

As $B \in$ P, there is a polynomial-time 1-DTM that decides it, which may be converted into a polynomial-time uniform family $\{C_n \,:\, n \in \mathbb{N}\}$ of Boolean circuits deciding B, as shown in Theorem 3.11 of Chapter 3. For a given input length $n \in \mathbb{N}$, every encoded pair $\langle x, y\rangle$ with $x \in \Sigma^n$ and $y \in \{0,1\}^p$ has the same length, which is given by a polynomial resource bound $q = q(n)$. Thus, the single circuit $C_{q(n)}$ decides membership in B for all such pairs $\langle x, y\rangle$.

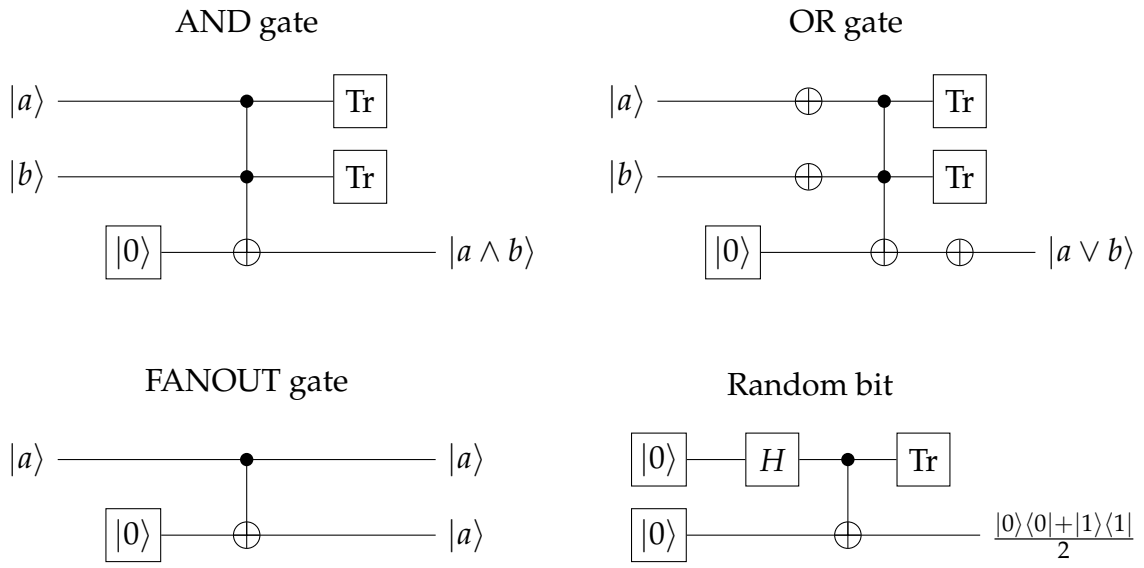

Figure 5.1: Simulations of the AND, OR, and FANOUT gates using Toffoli, CNOT, and NOT gates, together with a random-bit generator using Hadamard and CNOT gates. Ancillary and erasure gates are also used as shown.

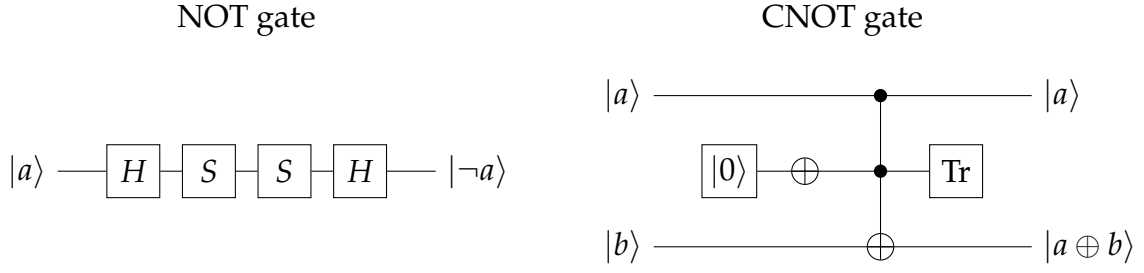

Figure 5.2: Simulations of the NOT and controlled-NOT gates over the standard gate set. On the left, $HS^2H = HZH = X$. On the right, an ancillary gate and a NOT gate fix the second control qubit of a Toffoli gate to $|1\rangle$, so that the Toffoli gate acts as a controlled-NOT gate on the remaining two qubits; the fixed qubit is discarded afterwards.

Now, for every $x \in \Sigma^*$, define a quantum circuit Q_x that uses the random-bit construction of Figure 5.1 to generate $y \in \{0,1\}^p$ uniformly at random, hard-codes the string x into $\langle x, y \rangle$ (using ancillary gates and NOT gates to flip $|0\rangle$ to $|1\rangle$ as needed), and simulates C_q on the result, using the constructions of Figures 5.1 and 5.2 to do this. The output qubit of the simulated circuit is taken as the output qubit of Q_x, and erasure gates discard everything else.

The simulated Boolean gates take standard basis states to standard basis states, and the random-bit construction produces a uniformly distributed classical bit, so the qubits of Q_x remain in probabilistic mixtures of standard basis states throughout the computation. The probability that Q_x accepts is precisely the fraction of strings $y \in \{0,1\}^p$ for which $\langle x, y \rangle \in B$. Therefore,

$$x \in A_{\text{yes}} \implies \Pr(Q_x \text{ accepts}) \geq 2/3, \tag{5.9}$$

$$x \in A_{\text{no}} \implies \Pr(Q_x \text{ accepts}) \leq 1/3, \tag{5.10}$$

as required.

Finally, the family $\{Q_x : x \in \Sigma^*\}$ is polynomial-time uniform, given that this is true of $\{C_n : n \in \mathbb{N}\}$, along with the fact that the bits representing x in any pair $\langle x, y \rangle$ can be easily computed from x itself, and the replacement of each Boolean gate in C_q is a simple, local substitution. Thus, $A \in \text{BQP}$. □

5.2 Error reduction

Now let us discuss error reduction for BQP computations. The method is the one used for BPP in Chapter 2: run several independent copies of the computation and compare the frequency of accepting outcomes against the midpoint of the two probabilities being distinguished. That argument is not specific to classical probabilistic computations — the amplification lemma (Lemma 2.14) is a statement about independent random variables with no stipulations on where they came from. Thus, the same argument applies here without modification. Figure 5.3 illustrates the resulting circuit when a majority vote is taken, which implicitly includes a measurement of its input qubits.

The theorem that follows allows the completeness and soundness probabilities to be functions of the input length rather than constants, provided these functions can be computed efficiently. We will say that a function $\alpha : \mathbb{N} \to [0,1]$ is *polynomial-time computable* if the mapping $0^n \mapsto \alpha(n)$ can be computed in polynomial time, where the value $\alpha(n)$ is understood to be a rational number, expressed as a pair of

integers (numerator and denominator) written in binary notation. Taking the input in unary notation like this is a formal way of expressing that $\alpha(n)$ is computable in time polynomial in n, which corresponds to the input length in the current setting. In contrast, if the input n was given in binary, then $\alpha(n)$ would need to be computed in time poly-logarithmic in n. This does not even leave enough time to write down a value that is exponentially small in n. We have seen this idea before, in the definition of polynomial resource bounds (Section 2.2).

Theorem 5.5 (Error reduction for BQP). *Let $\alpha, \beta : \mathbb{N} \to [0,1]$ be polynomial-time computable functions for which there exists a polynomial resource bound q satisfying*

$$2^{-q(n)} \leq \beta(n) \leq \alpha(n) \leq 1 - 2^{-q(n)} \quad \text{and} \quad \alpha(n) - \beta(n) \geq \frac{1}{q(n)}, \tag{5.11}$$

for all $n \in \mathbb{N}$. It follows that $\mathrm{BQP}(\alpha, \beta) = \mathrm{BQP}$.

Proof. Throughout the proof we will write $\alpha = \alpha(n)$, $\beta = \beta(n)$, $q = q(n)$, and so on, taking the argument n as implicit for clarity.

We first prove that

$$\mathrm{BQP}(\alpha, \beta) \subseteq \mathrm{BQP}\bigl(1 - 2^{-r}, 2^{-r}\bigr) \tag{5.12}$$

for every polynomial resource bound r, which requires only the last of the inequalities in (5.11). Suppose $A \in \mathrm{BQP}(\alpha, \beta)$, and take $\{Q_x\}$ to be a polynomial-time uniform family as in Definition 5.3. Let $N = 2rq^2$, and define a new family $\{R_x\}$, in which each circuit R_x runs N independent copies of Q_x and outputs 1 if and only if at least $\frac{\alpha+\beta}{2} \cdot N$ of the independent copies of Q_x output 1. This family is polynomial-time uniform: each R_x is assembled from N copies of Q_x, where N is a polynomial resource bound, and the polynomial-time computability of α and β allows for the generation of a circuit for the threshold value computation.

For each $k \in \{1, \ldots, N\}$, let X_k be the random variable taking the value 1 if the k-th copy produces the outcome 1 and the value 0 otherwise. These are independent random variables, each having expectation equal to the probability that Q_x accepts x. As $\alpha - \beta \geq 1/q$, we have $N \geq 2r/(\alpha - \beta)^2$, and therefore Lemma 2.14 implies that R_x produces an incorrect answer with probability at most 2^{-r}, in both the yes and no cases. This establishes (5.12).

Taking $r = 3$ in (5.12) now gives

$$\mathrm{BQP}(\alpha, \beta) \subseteq \mathrm{BQP}\bigl(\tfrac{7}{8}, \tfrac{1}{8}\bigr) \subseteq \mathrm{BQP}, \tag{5.13}$$

where the second containment holds trivially because $7/8 \geq 2/3$ and $1/8 \leq 1/3$. For the reverse containment, note that (5.12) requires only that $\alpha - \beta$ is at least the

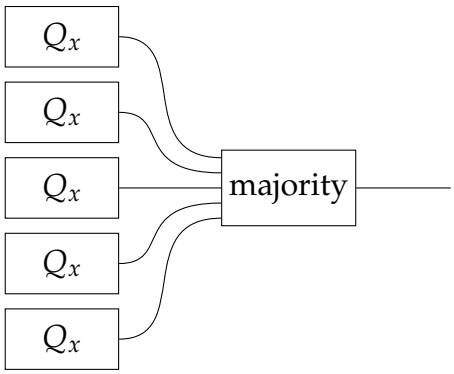

Figure 5.3: Error reduction for BQP by parallel repetition. Several independent copies of the circuit Q_x are run in parallel (five in this illustration), and their single-qubit outputs are combined via a majority gate to produce the final answer.

reciprocal of a polynomial resource bound, so it holds when $\alpha = 2/3$ and $\beta = 1/3$. Taking $r = q$ in this case gives

$$\mathrm{BQP} \subseteq \mathrm{BQP}\big(1 - 2^{-q},\, 2^{-q}\big) \subseteq \mathrm{BQP}(\alpha, \beta), \tag{5.14}$$

where the second containment follows from $\alpha \leq 1 - 2^{-q}$ and $\beta \geq 2^{-q}$. □

Because we were brief when introducing the classes BPP and PP in Chapter 2, it is appropriate at this point to reflect briefly on the importance of bounded error for error reduction. In particular, PP leaves no gap between the yes and no cases, and without an inverse polynomial gap the standard majority-of-independent-copies argument we just discussed fails: error reduction does not work for PP. Specifically, without an inverse polynomial lower-bound on ε in Hoeffding's inequality, the error bound $e^{-2N\varepsilon^2}$ becomes effectively worthless when N is polynomial.

5.3 Equivalent definitions

Circuits receive the input string

One can, alternatively, define BQP using the more traditional notion of circuit uniformity, though this only works for binary string inputs. For a polynomial-time uniform family $\{Q_n : n \in \mathbb{N}\}$ of quantum circuits, each outputting a single qubit, we say that Q_n accepts or rejects a binary string $x \in \{0,1\}^n$ if a standard basis

measurement of its output qubit on input $|x\rangle$ is 1 or 0, respectively. That is,

$$\Pr(Q_n \text{ accepts } x) = \langle 1|Q_n(|x\rangle\langle x|)|1\rangle, \tag{5.15}$$

$$\Pr(Q_n \text{ rejects } x) = \langle 0|Q_n(|x\rangle\langle x|)|0\rangle, \tag{5.16}$$

for every $x \in \{0,1\}^n$.

Definition 5.6 (Alternative definition of BQP)**.** Let $A = (A_{\text{yes}}, A_{\text{no}})$ be a promise problem over the binary alphabet and let $\alpha, \beta : \mathbb{N} \to [0,1]$ be functions (or constants). Then $A \in \text{BQP}(\alpha, \beta)$ if there exists a polynomial-time uniform family $\{Q_n : n \in \mathbb{N}\}$ of quantum circuits, where each Q_n has n input qubits and one output qubit, such that

$$x \in A_{\text{yes}} \implies \Pr(Q_{|x|} \text{ accepts } x) \geq \alpha(|x|), \tag{5.17}$$

$$x \in A_{\text{no}} \implies \Pr(Q_{|x|} \text{ accepts } x) \leq \beta(|x|). \tag{5.18}$$

Again, the complexity class BQP is defined as $\text{BQP} = \text{BQP}(2/3, 1/3)$.

Based on this definition, we can define BQP for promise problems over non-binary alphabets by encoding input strings as binary strings. We will make use of this definition later in the course, but Definition 5.3 is generally the easier of the two to use.

We will not go through a proof of the equivalence of these definitions aside from suggesting the main idea. One direction is straightforward: given a promise problem in BQP according to the alternative definition, we conclude that this problem is in BQP with respect to the first definition by replacing the input qubits with ancillary gates, along with NOT gates to flip $|0\rangle$ to $|1\rangle$ as needed.

The other direction is much more technical but still conceptually straightforward. For a given polynomial-time uniform family $\{R_x : x \in \Sigma^*\}$ for a promise problem $A = (A_{\text{yes}}, A_{\text{no}})$ in BQP, we may define a quantum circuit Q_n for each n that includes within it the polynomial-time computation that first obtains an encoding of R_x from the input x, and then executes this circuit. The first step can be done through the tableau construction of a Boolean circuit that simulates a 1-DTM on fixed-length inputs, followed by the creation of a "universal" quantum circuit that implements a quantum circuit having a bounded number of gates given its description. This is indeed possible — but you might not want to be responsible for implementing a circuit like this.

Sometimes it is convenient to work with a variation on the definition of BQP in which every circuit in the family is a *unitary* quantum circuit, the input state

is prepared as $|x, 0^{p(|x|)}\rangle$, and a single designated output qubit (typically the first qubit) is measured afterwards. The following simple proposition asserts that this restriction gives the same class. Here, $\Pr(Q_n \text{ accepts } x)$ is the probability that a standard basis measurement of the designated output qubit after applying Q_n to $|x, 0^{p(n)}\rangle$ gives the outcome 1, for each $x \in \{0,1\}^n$.

Proposition 5.7. *A promise problem A over the binary alphabet belongs to* BQP *if and only if there exist a polynomial resource bound p and a polynomial-time uniform family $\{Q_n : n \in \mathbb{N}\}$ of unitary quantum circuits, where each Q_n acts on $n + p(n)$ qubits, such that*

$$x \in A_{\text{yes}} \implies \Pr(Q_{|x|} \text{ accepts } x) \geq 2/3, \tag{5.19}$$

$$x \in A_{\text{no}} \implies \Pr(Q_{|x|} \text{ accepts } x) \leq 1/3. \tag{5.20}$$

This equivalence can be established through the unitary dilation construction from Chapter 4, applied to the alternative definition of BQP. Gathering all the ancillary input qubits into a single register initialized to $|0^{p(n)}\rangle$ at the start produces a unitary quantum circuit on $n + p(n)$ qubits whose designated output qubit has the same acceptance behaviour as the original circuit.

Circuits with other gate sets

Definition 5.3 assumes that quantum circuits are composed of gates drawn from our standard gate set, but sometimes other gate sets are of interest. To prove that a given gate set is universal can be challenging, as it requires showing that this gate set is capable of performing general quantum computations (for example, by showing that it can simulate gates from a known universal gate set). In contrast, the theorem that follows establishes that a broad collection of quantum gates gives no *additional* computational power beyond BQP.

The specific condition the theorem requires is a mild one on the matrix entries of the gates: each entry must be *polynomial-time weakly approximable*. To say that a complex number γ is polynomial-time weakly approximable means that there exists a polynomial-time DTM that, given 0^n as input for any $n \in \mathbb{N}$, outputs a Gaussian rational number γ_n (i.e., a complex number with rational real and imaginary parts) satisfying

$$|\gamma - \gamma_n| \leq \frac{1}{n}. \tag{5.21}$$

Numbers such as $1/\sqrt{2}$, $\cos(\pi/7)$, and any other algebraic number, as well as $e^{i\theta}$ for algebraic θ, are all polynomial-time weakly approximable, as are sums and products of other polynomial-time weakly approximable numbers.

It is instructive to split the argument that gates with polynomial-time weakly approximable entries stay inside BQP into two parts. First is a proposition showing that a single such gate can be efficiently approximated to any inverse-polynomial precision by a standard-gate-set circuit. The theorem itself then follows by applying the proposition to each unitary gate of a given family of quantum circuits, observing that approximation errors accumulate additively under composition.

Proposition 5.8. *Let U be a unitary operation on a constant number of qubits, and suppose that every entry of U is a polynomial-time weakly approximable complex number. For every polynomial resource bound q, there is a polynomial-time DTM that, on input 0^n, outputs the encoding of a standard-gate-set circuit implementing a unitary operation V satisfying*

$$\delta(V, U) \leq \frac{1}{q(n)}. \tag{5.22}$$

This proposition can be proved in two steps. First, using the polynomial-time weak approximability of the entries of U, we may obtain a description of a unitary operation W, computable in time polynomial in n, that satisfies $\delta(W, U) \leq 1/(2q(n))$. (This requires not only approximating the entries of U, but also adjusting the resulting matrix so that W is truly unitary.) Second, Theorem 4.4 may be applied in an effective form to obtain, in time polynomial in n, a standard-gate-set circuit implementing a unitary operation V with $\delta(V, W) \leq 1/(2q(n))$. By the triangle inequality, $\delta(V, U) \leq 1/q(n)$, as required.

Theorem 5.9. *Let $\mathcal{G}$ be a finite set of unitary quantum gates, and suppose that every entry of every gate in $\mathcal{G}$ is a polynomial-time weakly approximable complex number. Let $A = (A_{\text{yes}}, A_{\text{no}})$ be a promise problem for which there exists a polynomial-time uniform family $\{Q_x : x \in \Sigma^*\}$ of quantum circuits as in Definition 5.3, except that each unitary gate of each Q_x is drawn from $\mathcal{G}$ in place of the standard gate set, with ancillary and erasure gates allowed as before, such that*

$$x \in A_{\text{yes}} \implies \Pr(Q_x \text{ accepts}) \geq 2/3, \tag{5.23}$$

$$x \in A_{\text{no}} \implies \Pr(Q_x \text{ accepts}) \leq 1/3. \tag{5.24}$$

Then $A \in$ BQP.

We will not go through a proof of this theorem here, but only sketch the main idea. Given a family $\{Q_x\}$ as in the theorem, each unitary gate of Q_x is replaced by the standard-gate-set circuit given by Proposition 5.8, while the ancillary and erasure gates are left alone. Approximation errors accumulate additively under channel compositions, and the δ-distance between two operations is unchanged when

both are tensored with the identity on any number of additional qubits they do not touch. Therefore, if each unitary gate is approximated to δ-distance $1/q(n)$, where $n = |x|$, the channel implemented by the composite circuit differs from that of Q_x by at most $\text{size}(Q_x)/q(n)$ in δ-distance. By the polynomial-time uniformity of the family $\{Q_x\}$, we have that $\text{size}(Q_x)$ is polynomially bounded. Choosing q appropriately makes this discrepancy at most a constant $\eta < 1/6$, so that the resulting family establishes that A is in $\text{BQP}(2/3 - \eta, 1/3 + \eta) = \text{BQP}$ by Theorem 5.5.

Notice that the notion of polynomial-time weak approximability is (true to its name) weaker than the more standard *polynomial-time approximable* notion for real (or complex) numbers. That notion requires the DTM to output γ_n satisfying

$$|\gamma - \gamma_n| \leq 2^{-n} \tag{5.25}$$

on input 0^n, which is exponentially more precise than the $1/n$ error-bound demanded here. The weaker definition does capture more numbers, but in actuality most amplitudes a reader is likely to encounter (e.g., algebraic numbers and standard transcendental functions of them) are polynomial-time approximable in the stronger sense.

Correspondingly, the quantitative universality claim invoked in the proof of Proposition 5.8 is a weakening of Theorem 4.4, which produces a standard-gate-set circuit of size polynomial in $\log(1/\varepsilon)$ approximating a given constant-qubit unitary channel to δ-distance ε. Theorem 4.4 does yield a more efficient standard-gate-set simulation of each Q_x, but the conclusion of Theorem 5.9 requires only the weaker form, which (unlike Theorem 4.4) does not rely on the Solovay–Kitaev theorem (Theorem 4.5).

Chapter 6

BQP is contained in PP

The main goal of this chapter is to prove BQP $\subseteq$ PP. In words, any promise problem that can be solved in quantum polynomial time with bounded error can also be solved in probabilistic polynomial time with unbounded error. A reasonable take-away from this containment is that unbounded-error probabilistic polynomial time is very powerful (as opposed to bounded-error quantum polynomial time being weak). Indeed, we already have a sense that unbounded-error probabilistic polynomial time is powerful from the fact that NP $\subseteq$ PP.

The proof of the containment BQP $\subseteq$ PP that we will cover in this chapter is not necessarily the shortest or fastest proof, but it has the advantage of providing powerful technical machinery that will find additional uses later in the course.

6.1 Gap-definable functions and properties

We will begin by defining two classes of functions from strings to integers, which are commonly referred to as *counting classes* because they involve counting the number of strings that satisfy certain computational conditions. The first is called #P (typically read as "sharp P"), which was introduced by Valiant in 1979, and the second is called GapP, which was introduced by Fenner, Fortnow, and Kurtz in 1994. The two classes have a simple relationship — GapP functions are simply differences of #P functions — but it is GapP that will be most useful for our purposes.

We then establish several strong closure properties of GapP, which allow us to reason that many interesting functions are in this class. By applying what we learn to quantum circuits, together with a simple GapP characterization of PP, we will conclude that every promise problem in BQP is also in PP.

Counting complexity classes

We start with a definition of #P, which is as follows.

Definition 6.1. For a given alphabet Σ, a function $f : \Sigma^* \to \mathbb{N}$ is in #P if there exist a polynomial resource bound p and a language $B \in \mathrm{P}$ such that

$$f(x) = \big|\{y \in \{0,1\}^p \,:\, \langle x, y\rangle \in B\}\big| \tag{6.1}$$

for every $x \in \Sigma^*$.

Thus, in terms of the quantifier framework we used to define NP in Chapter 2, a #P function represents a count of the number of strings $y \in \{0,1\}^p$ that serve as a valid witness for a given string x to be a yes-input for an NP problem. Note that it is straightforward to use this definition to give alternative definitions for NP, BPP, and PP in terms of #P functions. For example, a promise problem $A = (A_{\mathrm{yes}}, A_{\mathrm{no}})$ is in NP if and only if there exists $f \in$ #P such that $f(x) \neq 0$ for all $x \in A_{\mathrm{yes}}$ and $f(x) = 0$ for all $x \in A_{\mathrm{no}}$.

The following proposition is simple but will nevertheless be quite useful going forward.

Proposition 6.2. *Suppose $f : \Sigma^* \to \mathbb{N}$ is in #P and $g : \Sigma^* \to \Sigma^*$ is polynomial-time computable. Then $f \circ g \in$ #P.*

Proof. By Definition 6.1, there exist a polynomial resource bound p and a language $B \in \mathrm{P}$ for which (6.1) is true for every $x \in \Sigma^*$. As g is polynomial-time computable, there must exist a polynomial resource bound q such that $|g(x)| \leq q(|x|)$ for every string $x \in \Sigma^*$. We note that, because p is a polynomial resource bound, it must be non-decreasing, and therefore $p(|g(x)|) \leq p(q(|x|))$ for all $x \in \Sigma^*$.

Define a language

$$C = \big\{\langle x, y0^k\rangle \,:\, \langle g(x), y\rangle \in B,\ |y| = p(|g(x)|),\ k = p(q(|x|)) - |y|\big\}, \tag{6.2}$$

which is in P by the assumption that $B \in \mathrm{P}$, g is polynomial-time computable, and p and q are polynomial resource bounds. It is the case that

$$\begin{aligned} &\big|\{z \in \{0,1\}^{p\circ q} \,:\, \langle x, z\rangle \in C\}\big| \\ &\quad = \big|\{y \in \{0,1\}^{p(|g(x)|)} \,:\, \langle g(x), y\rangle \in B\}\big| = f(g(x)), \end{aligned} \tag{6.3}$$

and therefore $f \circ g \in$ #P. □

Remark 6.3. It should be acknowledged that an analogous claim with the order of composition reversed is unlikely to be true. In particular, if $g \circ f \in \#\mathrm{P}$ for every #P function $f : \Sigma^* \to \mathbb{N}$ and every polynomial-time computable function of the form $g : \mathbb{N} \to \mathbb{N}$ (with inputs and outputs encoded in binary notation), then NP is closed under complementation (i.e., NP = coNP), which is widely conjectured to be false.

As the following proposition establishes, every function $f : \Sigma^* \to \mathbb{N}$ that is computable in polynomial time is contained in #P. As a point of clarity, a function f of this form is said to be polynomial-time computable if there exists a polynomial-time DTM that outputs $f(x)$ written in binary notation for each $x \in \Sigma^*$. Note that this necessarily implies that there exists a polynomial resource bound q such that $f(x) < 2^q$ for every $x \in \Sigma^*$, as a DTM cannot output more bits than the number of steps for which it runs.

Proposition 6.4. *Every polynomial-time computable function $f : \Sigma^* \to \mathbb{N}$ is in* #P.

Proof. Let q be a polynomial resource bound such that $f(x) < 2^q$ for every $x \in \Sigma^*$. Define

$$B = \big\{ \langle x, y \rangle \,:\, y \in \{0,1\}^q,\, (y)_2 < f(x) \big\}, \tag{6.4}$$

where $(y)_2 \in \mathbb{N}$ refers to the natural number encoded by y in binary notation. Because f is polynomial-time computable and natural numbers can be compared in polynomial (in fact, linear) time, we have $B \in \mathrm{P}$. Moreover,

$$\big| \{ y \in \{0,1\}^q \,:\, \langle x, y \rangle \in B \} \big| = f(x), \tag{6.5}$$

so $f \in \#\mathrm{P}$. □

Next we define the class of GapP functions, which (as mentioned above) are simply *differences* between #P functions. While the definition is very simple in conceptual terms, the resulting class of functions turns out to have remarkable closure properties, some of which will be discussed in the next section. The class GapP is extremely useful for reasoning about PP and other complexity classes connected with counting.

Definition 6.5. A function $g : \Sigma^* \to \mathbb{Z}$ is in GapP if there exist $f_0, f_1 \in \#\mathrm{P}$ such that $g = f_0 - f_1$.

One immediate consequence of the definition is that GapP is closed under negation: if $g \in \mathrm{GapP}$, then there exist $f_0, f_1 \in \#\mathrm{P}$ such that $g = f_0 - f_1$, and therefore $-g = f_1 - f_0 \in \mathrm{GapP}$. Another simple observation that will be useful later is

that for every GapP function g, the absolute value $|g(x)|$ is at most exponentially large in $|x|$. Specifically, if $g = f_0 - f_1$ with $f_0, f_1 \in$ #P and p is a polynomial resource bound with $f_0(x), f_1(x) \leq 2^p$ (as guaranteed by the definition of #P), then $|g(x)| \leq 2^p$ for every $x \in \Sigma^*$.

The following proposition follows immediately from Proposition 6.2 together with the definition of GapP. We will sometimes apply this proposition implicitly (to handle straightforward processing of arguments to GapP functions, for instance).

Proposition 6.6. *Suppose $f : \Sigma^* \to \mathbb{Z}$ is in* GapP *and $g : \Sigma^* \to \Sigma^*$ is polynomial-time computable. Then $f \circ g \in$* GapP*.*

To reason about additional properties of GapP, the proposition that follows is useful. In some sense, it provides a sort of *normal form* for GapP functions that is more convenient to work with than the definition in the context of some proofs.

As a reminder, when we say that a function h that has two string arguments is computable in polynomial time, we mean that there is a polynomial-time computable function that maps $\langle x, y \rangle \mapsto h(x, y)$ for all strings x and y, bearing in mind that we have selected a pairing function for which $\langle x, y \rangle$ is always a binary string, regardless of whatever (fixed) alphabets the strings x and y are drawn from.

Proposition 6.7. *A function $g : \Sigma^* \to \mathbb{Z}$ is in* GapP *if and only if there exist a polynomial resource bound p and a polynomial-time computable function*

$$h : \Sigma^* \times \{0,1\}^* \to \{-1, 0, +1\} \tag{6.6}$$

such that

$$g(x) = \sum_{y \in \{0,1\}^p} h(x, y) \tag{6.7}$$

for every $x \in \Sigma^*$.

Proof. Suppose first that $g \in$ GapP, so $g = f_0 - f_1$ for $f_0, f_1 \in$ #P. Thus, there exist polynomial resource bounds p_0 and p_1 and languages $B_0, B_1 \in$ P such that

$$f_a(x) = \big|\{z \in \{0,1\}^{p_a} \,:\, \langle x, z \rangle \in B_a\}\big| \tag{6.8}$$

for every $x \in \Sigma^*$ and $a \in \{0,1\}$.

To combine f_0 and f_1 into a single signed sum, we need a common witness length. Specifically, we will take $p(n) = p_0(n) + p_1(n)$, which is a polynomial

resource bound, and pad each witness for B_a with trailing zeros to length p by defining

$$C_0 = \{\langle x, z\,0^{p_1}\rangle : x \in \Sigma^*,\ z \in \{0,1\}^{p_0},\ \langle x, z\rangle \in B_0\}, \tag{6.9}$$

$$C_1 = \{\langle x, z\,0^{p_0}\rangle : x \in \Sigma^*,\ z \in \{0,1\}^{p_1},\ \langle x, z\rangle \in B_1\}. \tag{6.10}$$

(We could alternatively take $p(n) = \max\{p_0(n), p_1(n)\}$ and adjust the amount of padding.) Both C_0 and C_1 are in P, given that B_0 and B_1 are in P, and we have

$$f_a(x) = \left|\{y \in \{0,1\}^p \ :\ \langle x, y\rangle \in C_a\}\right| \tag{6.11}$$

for both $a \in \{0,1\}$.

Now define $h : \Sigma^* \times \{0,1\}^* \to \{-1, 0, +1\}$ as

$$h(x,y) = \begin{cases} +1 & \langle x, y\rangle \in C_0 \setminus C_1 \\ 0 & \langle x, y\rangle \in C_0 \cap C_1 \text{ or } \langle x, y\rangle \notin C_0 \cup C_1 \\ -1 & \langle x, y\rangle \in C_1 \setminus C_0 \end{cases} \tag{6.12}$$

which may alternatively be written $h = h_0 - h_1$ where h_0 and h_1 are the characteristic functions of C_0 and C_1, respectively. As $C_0, C_1 \in \mathrm{P}$, the function h is polynomial-time computable. Summing over witnesses of length p gives

$$\sum_{y \in \{0,1\}^p} h(x,y) = f_0(x) - f_1(x) = g(x), \tag{6.13}$$

as required.

For the reverse direction, suppose

$$g(x) = \sum_{y \in \{0,1\}^p} h(x,y) \tag{6.14}$$

for a polynomial-time computable function $h : \Sigma^* \times \{0,1\}^* \to \{-1, 0, +1\}$ and a polynomial resource bound p. Define languages

$$B_0 = \{\langle x, y\rangle \ :\ x \in \Sigma^*,\ y \in \{0,1\}^p,\ h(x,y) = +1\}, \tag{6.15}$$

$$B_1 = \{\langle x, y\rangle \ :\ x \in \Sigma^*,\ y \in \{0,1\}^p,\ h(x,y) = -1\}, \tag{6.16}$$

both of which are in P. The corresponding counting functions

$$f_a(x) = \left|\{y \in \{0,1\}^p \ :\ \langle x, y\rangle \in B_a\}\right| \tag{6.17}$$

for $a \in \{0,1\}$ are therefore in #P, and

$$f_0(x) - f_1(x) = \sum_{y \in \{0,1\}^p} h(x,y) = g(x), \tag{6.18}$$

showing that $g \in \mathrm{GapP}$. □

GapP characterization of PP

The last thing we will note about GapP functions before turning to their closure properties is the following simple characterization of PP.

Proposition 6.8. *A promise problem $A = (A_{\text{yes}}, A_{\text{no}})$ is in* PP *if and only if there exists $g \in$ GapP such that*

$$x \in A_{\text{yes}} \implies g(x) > 0, \tag{6.19}$$

$$x \in A_{\text{no}} \implies g(x) < 0. \tag{6.20}$$

Proof. Suppose first that $A = (A_{\text{yes}}, A_{\text{no}}) \in$ PP, so there exist a polynomial resource bound p and a language $B \in$ P such that the #P function

$$f(x) = \big|\{y \in \{0,1\}^p \,:\, \langle x, y\rangle \in B\}\big| \tag{6.21}$$

satisfies

$$x \in A_{\text{yes}} \implies f(x) > \tfrac{1}{2} \cdot 2^p \quad \text{and} \quad x \in A_{\text{no}} \implies f(x) < \tfrac{1}{2} \cdot 2^p. \tag{6.22}$$

Define

$$h(x) = \big|\{y \in \{0,1\}^p \,:\, \langle x, y\rangle \notin B\}\big| = 2^p - f(x), \tag{6.23}$$

which is also in #P because P is closed under complementation. The GapP function $g = f - h$ therefore satisfies $g(x) > 0$ for $x \in A_{\text{yes}}$ and $g(x) < 0$ for $x \in A_{\text{no}}$.

For the reverse implication, suppose $g \in$ GapP satisfies (6.19) and (6.20). By Proposition 6.7, there exist a polynomial resource bound p and a polynomial-time computable function $h : \Sigma^* \times \{0,1\}^* \to \{-1, 0, +1\}$ with

$$g(x) = \sum_{y \in \{0,1\}^p} h(x, y). \tag{6.24}$$

Define $q(n) = p(n) + 1$ (which is a polynomial resource bound) and define the language

$$C = \big\{\langle x, ya\rangle \,:\, x \in \Sigma^*,\ y \in \{0,1\}^p,\ a \in \{0,1\},\ h(x,y) \geq a\big\}, \tag{6.25}$$

which is in P by the assumption that h is polynomial-time computable. For a given choice of $x \in \Sigma^*$ and $y \in \{0,1\}^p$, the number of binary values $a \in \{0,1\}$ for which $\langle x, ya\rangle \in C$ is $h(x,y) + 1$, and therefore

$$\big|\{z \in \{0,1\}^q \,:\, \langle x, z\rangle \in C\}\big| = \sum_{y \in \{0,1\}^p} \big(1 + h(x,y)\big) = \frac{1}{2} \cdot 2^q + g(x). \tag{6.26}$$

As $x \in A_{\text{yes}}$ implies $g(x) > 0$ and $x \in A_{\text{no}}$ implies $g(x) < 0$, we conclude that $A \in$ PP. □

Closure properties

We will prove two fundamental closure properties of GapP functions in this section: closure under *exponential sums* and *polynomial products*. These properties can then be combined to conclude that there is a GapP-function for the entries of a matrix obtained by multiplying together a polynomial number of exponentially large matrices, provided that they have entries described by a GapP function.

In the theorems that follow, our convention regarding functions taking multiple arguments extends naturally to GapP functions. Specifically, to say that

$$g : \Sigma^* \times \{0,1\}^* \to \mathbb{Z} \tag{6.27}$$

is a GapP function means that there is a GapP function taking the value $g(x,y)$ on input $\langle x,y\rangle$ for every $x \in \Sigma^*$ and $y \in \{0,1\}^*$.

Theorem 6.9. *Let $g : \Sigma^* \times \{0,1\}^* \to \mathbb{Z}$ be a* GapP *function and let p be a polynomial resource bound. The function $h : \Sigma^* \to \mathbb{Z}$ defined by*

$$h(x) = \sum_{y \in \{0,1\}^{p}} g(x,y) \tag{6.28}$$

is in GapP.

Proof. First let us observe that there exists a polynomial resource bound r such that $|\langle x,y\rangle| = r(|x|)$ for every pair of strings $x \in \Sigma^*$ and $y \in \{0,1\}^{p}$, given our pairing function selection and the fact that p is a polynomial resource bound. Thus, for any polynomial resource bound q, we have $q(|\langle x,y\rangle|) = q(r(|x|))$ for all $x \in \Sigma^*$ and $y \in \{0,1\}^{p}$.

By Proposition 6.7 there exist a polynomial resource bound q and a polynomial-time computable function $f : \{0,1\}^* \times \{0,1\}^* \to \{-1,0,+1\}$ such that

$$g(x,y) = \sum_{z \in \{0,1\}^{q \circ r}} f(\langle x,y\rangle, z). \tag{6.29}$$

The existence of a polynomial-time computable function

$$F : \Sigma^* \times \{0,1\}^* \to \{-1,0,+1\} \tag{6.30}$$

such that

$$F(x,yz) = f(\langle x,y\rangle, z) \tag{6.31}$$

for all $y \in \{0,1\}^{p}$ and $z \in \{0,1\}^{q \circ r}$ follows, and hence

$$h(x) = \sum_{w \in \{0,1\}^{s}} F(x,w) \tag{6.32}$$

for $s(n) = p(n) + q(r(n))$, which is a polynomial resource bound. By Proposition 6.7, we conclude that $h \in \mathrm{GapP}$. □

Theorem 6.10. *Let $g : \Sigma^* \times \mathbb{N} \to \mathbb{Z}$ be a* GapP *function and let p be a polynomial resource bound. The function $h : \Sigma^* \to \mathbb{Z}$ defined by*

$$h(x) = \prod_{k=1}^{p} g(x,k) \tag{6.33}$$

is in GapP.

Proof. The fact that k written in binary has variable length is a minor inconvenience that we will circumvent as follows: for every $m, k \in \mathbb{N}$, let $\mathrm{bin}(k,m) \in \{0,1\}^m$ denote the string representing the m least significant bits of the binary encoding of k, either truncating leading bits or adding leading 0s. For this proof, we will only need to add leading 0s, not truncate leading bits.

By the assumption that $g \in \mathrm{GapP}$, together with Proposition 6.6, we conclude that there must exist a GapP function $G : \Sigma^* \times \{0,1\}^* \to \mathbb{Z}$ such that

$$G(x, \mathrm{bin}(k,p)) = g(x,k) \tag{6.34}$$

for every $k < 2^p$, and hence for every $k \in \{1,\ldots,p\}$. Thus, by Proposition 6.7, there exist a polynomial resource bound q along with a polynomial-time computable function $f : \{0,1\}^* \times \{0,1\}^* \to \{-1,0,+1\}$ such that

$$g(x,k) = G(x, \mathrm{bin}(k,p)) = \sum_{z \in \{0,1\}^{q \circ r}} f(\langle x, \mathrm{bin}(k,p)\rangle, z) \tag{6.35}$$

for every $x \in \Sigma^*$ and $k \in \{1,\ldots,p\}$. Here, r is a polynomial resource bound satisfying $|\langle x,y\rangle| = r(|x|)$ for all $x \in \Sigma^*$ and $y \in \{0,1\}^p$.

Now define $F : \Sigma^* \times \{0,1\}^* \to \{-1,0,+1\}$ as

$$F(x, z_1 z_2 \cdots z_p) = \prod_{k=1}^{p} f\big(\langle x, \mathrm{bin}(k,p)\rangle, z_k\big) \tag{6.36}$$

for binary strings $z_1,\ldots,z_p$ each of length $q \circ r$. The function F takes values in $\{-1,0,+1\}$ (as it is a product of values in that set) and is polynomial-time computable. Distributing the product over the sums, we have

$$h(x) = \prod_{k=1}^{p} \sum_{z_k \in \{0,1\}^{q \circ r}} f\big(\langle x, \mathrm{bin}(k,p)\rangle, z_k\big) = \sum_{w \in \{0,1\}^s} F(x,w) \tag{6.37}$$

for $s(n) = p(n) \cdot q(r(n))$, which is a polynomial resource bound. It follows that $h \in \mathrm{GapP}$ by Proposition 6.7. □

Finally, here is the theorem suggested at the start of the subsection concerning polynomial products of matrices having entries described by a GapP function.

Theorem 6.11. *Let p and q be polynomial resource bounds, and suppose that, for every string $x \in \Sigma^*$ and every $k \in \{1, \ldots, p\}$, a matrix $A_{x,k}$ with integer entries is given, indexed by $\{0,1\}^q \times \{0,1\}^q$. If there exists a* GapP *function*

$$g : \Sigma^* \times \mathbb{N} \times \{0,1\}^* \times \{0,1\}^* \to \mathbb{Z} \tag{6.38}$$

such that

$$g(x,k,y,z) = \langle y | A_{x,k} | z \rangle \tag{6.39}$$

for all x, k, y, and z, then there exists a GapP *function*

$$G : \Sigma^* \times \{0,1\}^* \times \{0,1\}^* \to \mathbb{Z} \tag{6.40}$$

such that

$$G(x,y,z) = \langle y | A_{x,p} \cdots A_{x,1} | z \rangle \tag{6.41}$$

for all x, y, and z.

Proof. By Proposition 6.6 there exists a GapP function f such that

$$f(\langle x, w_0 \cdots w_p \rangle, k) = g(x,k,w_k,w_{k-1}) = \langle w_k | A_{x,k} | w_{k-1} \rangle \tag{6.42}$$

for every $x \in \Sigma^*$, $k \in \{1, \ldots, p\}$, and $w_0, \ldots, w_p \in \{0,1\}^q$. Hence, by Theorem 6.10, along with some routine manipulations of polynomial resource bounds, we may conclude that there exists a GapP function $h : \Sigma^* \times \{0,1\}^* \to \mathbb{Z}$ such that

$$h(x, w_0 \cdots w_p) = \prod_{k=1}^{p} f(\langle x, w_0 \cdots w_p \rangle, k) = \prod_{k=1}^{p} \langle w_k | A_{x,k} | w_{k-1} \rangle, \tag{6.43}$$

again for all $x \in \Sigma^*$ and $w_0, \ldots, w_p \in \{0,1\}^q$. By Theorem 6.9, we conclude that there exists a GapP function $G : \Sigma^* \times \{0,1\}^* \times \{0,1\}^* \to \mathbb{Z}$ such that

$$G(x,y,z) = \sum_{w_1 \cdots w_{p-1} \in \{0,1\}^{(p-1)q}} h(x, z w_1 \cdots w_{p-1} y) = \langle y | A_{x,p} \cdots A_{x,1} | z \rangle \tag{6.44}$$

as required. □

As simple special cases of these two theorems, we have that finite sums and products of GapP functions are also GapP functions — which are facts that could alternatively be proved directly. We will typically use these facts implicitly when we need them, without referring explicitly to one of the two theorems.

6.2 Connection to quantum circuits

Next we will apply the GapP machinery of the previous section to BQP, making use of the so-called *natural representation* of quantum channels, which allows us to describe their actions and compositions in a simple and direct way through matrix products.

The natural representation of a channel

The natural representation of a quantum channel is defined through the *vectorization* operation, which reshapes a matrix into a column vector. Specifically, given an $m \times n$ complex matrix M, its vectorization is

$$\operatorname{vec}(M) = \sum_{j=1}^{m} \sum_{k=1}^{n} \langle j|M|k\rangle \, |j\rangle \otimes |k\rangle \in \mathbb{C}^m \otimes \mathbb{C}^n. \tag{6.45}$$

Alternatively, vec is the linear mapping defined by the action

$$\operatorname{vec}(|j\rangle\langle k|) = |j\rangle|k\rangle \tag{6.46}$$

for all $j \in \{1, \ldots, m\}$ and $k \in \{1, \ldots, n\}$. For example, the single-qubit density matrix $|0\rangle\langle 0|$ has vectorization

$$\operatorname{vec}(|0\rangle\langle 0|) = |0\rangle \otimes |0\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \tag{6.47}$$

while the completely mixed state $\mathbb{1}/2$ has vectorization

$$\operatorname{vec}(\mathbb{1}/2) = \frac{1}{2}(|0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle) = \frac{1}{2} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}. \tag{6.48}$$

With that notation established, we can introduce the natural representation of a quantum channel as follows.

Definition 6.12. Let Φ be a channel from n qubits to m qubits. The *natural representation* of Φ is the unique $4^m \times 4^n$ complex matrix $K(\Phi)$ satisfying

$$K(\Phi) \operatorname{vec}(\rho) = \operatorname{vec}(\Phi(\rho)) \tag{6.49}$$

for every $2^n \times 2^n$ density matrix ρ.

It is called the *natural representation* because, when states are vectorized, this *is* the channel's matrix. Consequently, *compositions* of channels are given by ordinary matrix products of their natural representations.

Of course, the natural representation is really only as natural as the vectorization map is natural — but given a standard basis, the linear map defined by $|a\rangle\langle b| \mapsto |a\rangle|b\rangle$ for all standard basis vectors $|a\rangle$ and $|b\rangle$ is undeniably a natural way to vectorize. Confusingly, it is also common for authors to define the vectorization map by $|a\rangle\langle b| \mapsto |b\rangle|a\rangle$, so always check which definition is being used.

We can connect the natural representation of a channel to its Kraus representations as follows. For a channel Φ with Kraus operators $A_1, \ldots, A_r$, so that

$$\Phi(\rho) = \sum_{k=1}^{r} A_k \rho A_k^* \tag{6.50}$$

for every density matrix ρ, the natural representation of Φ is given by

$$K(\Phi) = \sum_{k=1}^{r} A_k \otimes \overline{A_k}, \tag{6.51}$$

where $\overline{A_k}$ denotes the entrywise complex conjugate of A_k. For a unitary matrix U, the channel $\rho \mapsto U\rho U^*$ associated with U has U as a single Kraus operator. Writing $K(U)$ as a shorthand for the natural representation of the channel described by U, this becomes

$$K(U) = U \otimes \overline{U}. \tag{6.52}$$

The natural representations of the gates in our standard gate set can be written down explicitly. For the Hadamard gate, the matrix H has real entries, so $\overline{H} = H$ and

$$K(H) = H \otimes H = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}. \tag{6.53}$$

For the S gate,

$$K(S) = S \otimes \overline{S} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -i & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{6.54}$$

For the TOF gate,

$$K(\text{TOF}) = \text{TOF} \otimes \text{TOF}, \tag{6.55}$$

which is a 64×64 permutation matrix. Its action on standard basis states is given by

$$K(\mathrm{TOF})\,|a_0, b_0, c_0, a_1, b_1, c_1\rangle = |a_0, b_0, (a_0 \wedge b_0) \oplus c_0, a_1, b_1, (a_1 \wedge b_1) \oplus c_1\rangle \tag{6.56}$$

for every $a_0, b_0, c_0, a_1, b_1, c_1 \in \{0, 1\}$.

The first of the two non-unitary gates in our standard gate set is the ancillary gate, which we denote as $|0\rangle$. As a channel, it maps scalars to 2×2 matrices as $\alpha \mapsto \alpha\,|0\rangle\langle 0|$. It is therefore correct to say that $|0\rangle$ is a single Kraus operator for this channel, which makes its natural representation a column vector. Specifically, writing $K(|0\rangle)$ as a shorthand for the natural representation of this channel, we have

$$K(|0\rangle) = |0\rangle \otimes |0\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}. \tag{6.57}$$

The second non-unitary gate is the erasure gate, which already goes by the name Tr when viewed as a channel, with the understanding that specifically this is the trace of a 2×2 matrix. One Kraus representation of this channel is obtained by taking Kraus operators $\langle 0|$ and $\langle 1|$, which gives

$$K(\mathrm{Tr}) = \langle 0| \otimes \langle 0| + \langle 1| \otimes \langle 1| = \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix}. \tag{6.58}$$

We could take any two orthonormal vectors in place of $|0\rangle$ and $|1\rangle$ to obtain a valid Kraus representation of this channel, but note that we always obtain the same natural representation — unlike Kraus representations, natural representations of channels are unique.

As was already mentioned, channel composition corresponds to matrix multiplication of their natural representations: for channels Φ and Ψ for which the composition $\Psi \circ \Phi$ makes sense, we have

$$K(\Psi \circ \Phi) = K(\Psi)\,K(\Phi). \tag{6.59}$$

This follows directly from Definition 6.12.

Here is a simple example of a channel composition expressed in terms of the natural representation. Assume U is a unitary operation on a single qubit, and write

$$U|0\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{6.60}$$

which implies

$$\overline{U}|0\rangle = \overline{\alpha}|0\rangle + \overline{\beta}|1\rangle. \tag{6.61}$$

Consider the composition of three gates: first an ancillary gate, which creates an initialized qubit; then a U gate, which takes the state to $\alpha|0\rangle + \beta|1\rangle$; and finally an erasure gate, which throws the qubit away and leaves us with the scalar 1 (i.e., nothing, as a density matrix). In terms of the natural representations of their corresponding channels, the composition is given by $K(\mathrm{Tr})\,K(U)\,K(|0\rangle)$. Let us multiply this product out, starting on the right, to verify this. First we have

$$K(U)\,K(|0\rangle) = (U \otimes \overline{U})(|0\rangle \otimes |0\rangle) = U|0\rangle \otimes \overline{U}|0\rangle = \begin{pmatrix} |\alpha|^2 \\ \alpha\overline{\beta} \\ \beta\overline{\alpha} \\ |\beta|^2 \end{pmatrix}. \tag{6.62}$$

Therefore

$$K(\mathrm{Tr})\,K(U)\,K(|0\rangle) = \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} |\alpha|^2 \\ \alpha\overline{\beta} \\ \beta\overline{\alpha} \\ |\beta|^2 \end{pmatrix} = |\alpha|^2 + |\beta|^2 = 1. \tag{6.63}$$

Measurements can also be described in a similar way. For instance, a standard basis measurement can be described by natural representations of the maps that effectively condition on the possible outcomes.

$$K(\langle 0|) = \langle 0| \otimes \langle 0| = \begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix} \tag{6.64}$$

$$K(\langle 1|) = \langle 1| \otimes \langle 1| = \begin{pmatrix} 0 & 0 & 0 & 1 \end{pmatrix} \tag{6.65}$$

Substituting these in turn for $K(\mathrm{Tr})$ yields the probabilities $|\alpha|^2$ and $|\beta|^2$ as expected.

GapP functions from quantum circuits

Now let us turn specifically to quantum circuits over our standard gate set, focusing on just the unitary gates H, S, and TOF. Let

$$\{U_x \,:\, x \in \Sigma^*\} \tag{6.66}$$

be a polynomial-time uniform family of unitary quantum circuits, over the gate set $\{H, S, \mathrm{TOF}\}$, such that each U_x acts on $q = q(|x|)$ qubits and consists of $s = s(|x|)$ gates, for polynomial resource bounds q and s. With respect to a fixed topological ordering of the gates of U_x, which can be computed in polynomial time from the

circuit's description, let $U_{x,k}$ be the unitary operation on q qubits describing the *global* action of gate number k of U_x. This means that each $U_{x,k}$ acts as the identity operation on all but at most three qubits.

The product of the natural representations of the corresponding channels,

$$K(U_x) = K(U_{x,s}) \cdots K(U_{x,1}), \tag{6.67}$$

is therefore of interest, as this is the natural representation of the channel computed by U_x. The rows and columns of these matrices are indexed by binary strings of length $2q$, which is polynomial in $|x|$, and there are s matrices in the product, which will allow us to apply the final theorem about GapP functions from the previous section.

Lemma 6.13. *Let $\{U_x : x \in \Sigma^*\}$ be a polynomial-time uniform family of unitary quantum circuits over the gate set $\{H, S, \mathrm{TOF}\}$, where each U_x has $s = s(|x|)$ gates and acts on $q = q(|x|)$ qubits, for polynomial resource bounds q and s. There exist* GapP *functions F and G such that*

$$F(x, y, z) + iG(x, y, z) = 2^s \langle y|K(U_x)|z\rangle \tag{6.68}$$

for every $x \in \Sigma^*$ and $y, z \in \{0,1\}^{2q}$.

Proof. First we will prove the lemma for the gate set $\{H, \mathrm{TOF}\}$. As these two gates have only real entries, G may be taken to be identically zero. Define

$$M_{x,k} = 2K(U_{x,k}) \tag{6.69}$$

for every $x \in \Sigma^*$ and $k \in \{1, \ldots, s\}$, where each $K(U_{x,k})$ is as described above. Each of these matrices has entries in the set $\{-1, 0, 1, 2\}$ that can be computed in polynomial time, and therefore there exists a GapP function

$$g : \Sigma^* \times \mathbb{N} \times \{0,1\}^* \times \{0,1\}^* \to \mathbb{Z} \tag{6.70}$$

such that

$$g(x, k, y, z) = \langle y|M_{x,k}|z\rangle \tag{6.71}$$

for every $x \in \Sigma^*$, $k \in \{1, \ldots, s\}$, and $y, z \in \{0,1\}^{2q}$. It follows from Theorem 6.11 that there exists a GapP function F satisfying

$$F(x, y, z) = \langle y|M_{x,s} \cdots M_{x,1}|z\rangle = 2^s \langle y|K(U_x)|z\rangle, \tag{6.72}$$

as required.

Now we will prove the lemma in general, where S gates may also be included in the circuit. Recall that the natural representation of an S gate is

$$K(S) = S \otimes \overline{S} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -i & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{6.73}$$

Defining each $M_{x,k}$ precisely as before in (6.69), we find that the entries of these matrices may now include $\pm 2i$ in addition to -1, 0, 1, and 2. We can handle the imaginary numbers with a simple and well-known trick.

The idea is to make use of the correspondence

$$A + iB \longleftrightarrow \begin{pmatrix} A & -B \\ B & A \end{pmatrix} \tag{6.74}$$

for real, square matrices A and B. Specifically, this correspondence respects matrix multiplication:

$$(A + iB)(C + iD) = (AC - BD) + i(AD + BC) \tag{6.75}$$

is consistent with

$$\begin{pmatrix} A & -B \\ B & A \end{pmatrix} \begin{pmatrix} C & -D \\ D & C \end{pmatrix} = \begin{pmatrix} AC - BD & -(AD + BC) \\ AD + BC & AC - BD \end{pmatrix} \tag{6.76}$$

for all real, square matrices A, B, C, and D of the same size. With this representation in mind, let $A_{x,k}$ and $B_{x,k}$ be the integer matrices for which $M_{x,k} = A_{x,k} + iB_{x,k}$, and define

$$R_{x,k} = \begin{pmatrix} A_{x,k} & -B_{x,k} \\ B_{x,k} & A_{x,k} \end{pmatrix} \tag{6.77}$$

for every $x \in \Sigma^*$ and $k \in \{1, \ldots, s\}$. The rows and columns of these matrices are indexed by $\{0,1\}^{2q+1}$, where the leading bits index one of the four blocks. The product $R_{x,s} \cdots R_{x,1}$ is therefore the 2×2 real block matrix corresponding to the product

$$M_{x,s} \cdots M_{x,1} = 2^s K(U_x). \tag{6.78}$$

Its diagonal blocks are therefore both equal to the real part of $2^s K(U_x)$, while the off-diagonal blocks are ± 1 times the imaginary part.

Finally, each $R_{x,k}$ has integer entries that can be computed in polynomial time, so by Theorem 6.11 there exists a GapP function h satisfying

$$h(x, u, v) = \langle u | R_{x,s} \cdots R_{x,1} | v \rangle \tag{6.79}$$

for every $x \in \Sigma^*$ and all $u, v \in \{0,1\}^{2q+1}$. By Proposition 6.6, the functions defined by

$$F(x,y,z) = h(x,0y,0z), \tag{6.80}$$

$$G(x,y,z) = h(x,1y,0z), \tag{6.81}$$

are therefore GapP functions satisfying (6.68). □

With that lemma in hand, we may state and prove the following theorem, which relates quantum circuits and GapP functions in a more convenient form.

Theorem 6.14. *Let $\{Q_x : x \in \Sigma^*\}$ be a polynomial-time uniform family of quantum circuits, where each Q_x has $p = p(|x|)$ input qubits and $r = r(|x|)$ output qubits, for polynomial resource bounds p and r. There exist* GapP *functions f and g and a polynomial resource bound s such that*

$$f(x,u,v,y,z) + ig(x,u,v,y,z) = 2^s \langle y|Q_x\big(|u\rangle\langle v|\big)|z\rangle \tag{6.82}$$

for every $x \in \Sigma^*$, all $u, v \in \{0,1\}^p$, and all $y, z \in \{0,1\}^r$.

Proof. By Proposition 5.2 there exist polynomial resource bounds q and s, together with a polynomial-time uniform family $\{U_x\}$ of unitary quantum circuits, such that each U_x is a unitary dilation of Q_x using exactly $q = q(|x|)$ ancillary qubits and having size exactly $s = s(|x|)$. Each U_x therefore acts on $p+q$ qubits, and after the circuit is run the first r qubits are taken as the output of Q_x and the remaining $p+q-r$ are garbage qubits. That is,

$$Q_x(\rho) = \big(\mathbb{I}^{\otimes r} \otimes \mathrm{Tr}^{\otimes(p+q-r)}\big)\Big(U_x\big(\rho \otimes |0^q\rangle\langle 0^q|\big)U_x^*\Big) \tag{6.83}$$

for every $2^p \times 2^p$ density matrix ρ.

At this point, we could proceed by obtaining the natural representation $K(Q_x)$ of each Q_x from $K(U_x)$ through matrix multiplications involving the natural representations of the non-unitary gates of Q_x (i.e., the ancillary and erasure gates). More directly (and equivalently), we may observe that

$$\langle y|Q_x\big(|u\rangle\langle v|\big)|z\rangle = \sum_{w \in \{0,1\}^{p+q-r}} \langle ywzw|K(U_x)|u0^q v0^q\rangle \tag{6.84}$$

for every $u, v \in \{0,1\}^p$ and $y, z \in \{0,1\}^r$. (The two 0^q portions of the standard basis states in the ket on the right-hand side correspond to the q ancillary gates, while the w portions in the bra, summed over all binary strings of length $p+q-r$, correspond to the erasure gates acting on the garbage qubits.)

By Lemma 6.13 there exist GapP functions F and G for the entries of $2^s K(U_x)$, as in (6.68), but where the strings indexing the entries have length $2(p+q)$. Define

$$f(x,u,v,y,z) = \sum_{w \in \{0,1\}^{p+q-r}} F\big(x,\ ywzw,\ u0^q v0^q\big), \tag{6.85}$$

$$g(x,u,v,y,z) = \sum_{w \in \{0,1\}^{p+q-r}} G\big(x,\ ywzw,\ u0^q v0^q\big). \tag{6.86}$$

As the two strings appearing as arguments of F and G are computable in polynomial time from x, u, v, y, z, and w, it follows that f and g are GapP functions by Proposition 6.6 and Theorem 6.9, along with some routine manipulations of polynomial resource bounds. Multiplying (6.84) through by 2^s reveals that f and g satisfy (6.82), as required. □

Let us observe the following two corollaries of this theorem. Both are nearly immediate special cases, isolated for convenience.

Corollary 6.15. *Let $\{Q_x : x \in \Sigma^*\}$ be a polynomial-time uniform family of quantum circuits, where each Q_x has no input qubits and $r = r(|x|)$ output qubits, for a polynomial resource bound r, and let ρ_x denote the state output by Q_x. There exist* GapP *functions f and g and a polynomial resource bound s such that*

$$f(x,y,z) + ig(x,y,z) = 2^s \langle y|\rho_x|z\rangle \tag{6.87}$$

for every $x \in \Sigma^*$ and all $y, z \in \{0,1\}^r$.

Proof. Take p to be identically zero in Theorem 6.14, so that u and v are both the empty string, which means $|u\rangle\langle v|$ is the scalar 1, giving $Q_x(|u\rangle\langle v|) = \rho_x$. Discarding the two empty-string arguments leaves GapP functions by Proposition 6.6. □

Corollary 6.16. *Let $\{Q_x : x \in \Sigma^*\}$ be a polynomial-time uniform family of quantum circuits, where each Q_x has no input qubits and one output qubit. There exist a* GapP *function h and a polynomial resource bound s such that*

$$h(x) = 2^s \Pr(Q_x \text{ accepts}) \tag{6.88}$$

for every $x \in \Sigma^*$.

Proof. Take f and s as in Corollary 6.15 for r identically equal to 1, and let $h(x) = f(x,1,1)$. The acceptance probability $\Pr(Q_x \text{ accepts}) = \langle 1|\rho_x|1\rangle$ is a real number, so $h(x) = 2^s \Pr(Q_x \text{ accepts})$ as required. □

The containment of BQP in PP

Now we can prove the main theorem of the chapter, allowing us to update our Hasse diagram to include BQP, as shown in Figure 6.1.

Theorem 6.17. BQP $\subseteq$ PP.

Proof. Let $A = (A_{\text{yes}}, A_{\text{no}}) \in \text{BQP}$ and let $\{Q_x : x \in \Sigma^*\}$ be a polynomial-time uniform family of quantum circuits as in Definition 5.3, so that each Q_x has no input qubits and one output qubit, and

$$x \in A_{\text{yes}} \implies \Pr(Q_x \text{ accepts}) \geq 2/3, \tag{6.89}$$

$$x \in A_{\text{no}} \implies \Pr(Q_x \text{ accepts}) \leq 1/3. \tag{6.90}$$

By Corollary 6.16 there exist a GapP function h and a polynomial resource bound s such that

$$h(x) = 2^s \Pr(Q_x \text{ accepts}) \tag{6.91}$$

for every $x \in \Sigma^*$. Define

$$f(x) = 2\,h(x) - 2^s, \tag{6.92}$$

which is also a GapP function. To see this, note that the function $x \mapsto 2^s$ is (rather trivially) a #P function, so $x \mapsto -2^s$ is a GapP function, and the closure of GapP under finite sums (which follows from Theorem 6.9 but can also be proved directly) implies $f \in \text{GapP}$. The sign of $f(x)$ clearly agrees with the sign of

$$\Pr(Q_x \text{ accepts}) - 1/2. \tag{6.93}$$

Because $2/3 > 1/2 > 1/3$, we have $f > 0$ on A_{yes} and $f < 0$ on A_{no}, so by Proposition 6.8 it follows that $A \in \text{PP}$. □

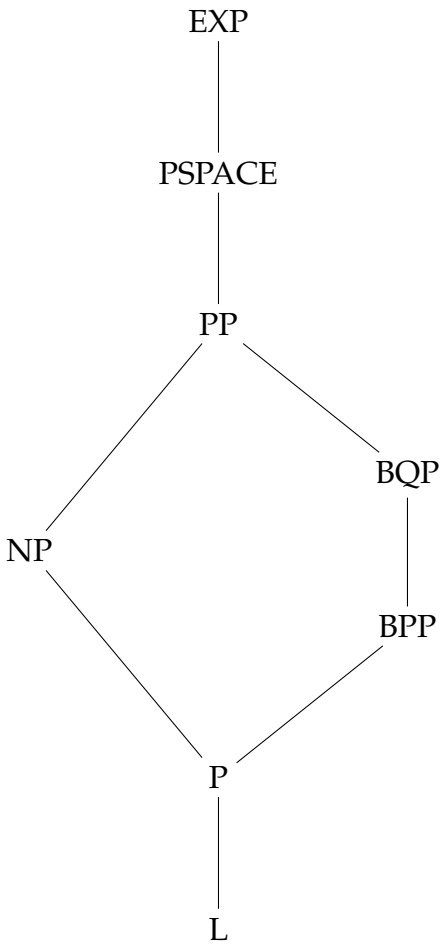

Figure 6.1: A Hasse diagram similar to Figure 2.1, extended to include BQP.

Chapter 7

BQP with postselection

This chapter focuses on the complexity class PostBQP, which is the class of promise problems that can be decided with bounded error by polynomial-time quantum algorithms that make use of *postselection*. Here, postselection refers to the process of disregarding the output of a computation when a certain condition is not met.

In greater detail, imagine a computation (which could, in general, be quantum or classical) that produces two output bits. The first output bit is the postselection bit, and when this bit is 0, the computation is deemed to have failed, and is therefore disregarded. On the other hand, if the postselection output bit is 1, the computation has succeeded and the second output bit is taken to indicate the result of the computation. In the specific case of PostBQP, the computation itself must be performed by a polynomial-time quantum computation (or, equivalently, by a polynomial-time uniform family of quantum circuits).

It is required that the postselection bit outputs 1 with a nonzero probability — so the probabilities of the second bit being 0 and 1 *conditioned* on the postselection bit being 1 are well-defined — though there is no requirement that the postselection bit outputs 1 with a nonnegligible probability. (The probability that the postselection bit outputs 1 could, for instance, be exponentially small.) In contrast, the probabilities that the second output bit is 0 or 1 conditioned on the postselection bit being 1 must obey the typical conditions of a bounded-error computation.

The fact that there is no requirement that postselection succeeds with a nonnegligible probability means that postselected computations should not be considered practical. PostBQP is nevertheless an interesting class from a theoretical perspective. Indeed, as we will prove in this chapter, it turns out that PostBQP $=$ PP. This is not only interesting in its own right, but is interesting because of what it says about the class PP. In particular, we may conclude from this characterization that,

as a class of languages, PP is closed under intersection:

$$A, B \in \mathrm{PP} \implies A \cap B \in \mathrm{PP}. \tag{7.1}$$

While this fact has been known since the early 1990s, a significantly simplified proof of it is revealed by the characterization PostBQP = PP.

7.1 Definition and basic properties

Postselected BQP

We will begin with the following formal definition of PostBQP.

Definition 7.1. Let $A = (A_{\mathrm{yes}}, A_{\mathrm{no}})$ be a promise problem over an alphabet Σ and let $\alpha, \beta : \mathbb{N} \to [0,1]$ be functions (or constants). Then $A \in \mathrm{PostBQP}(\alpha, \beta)$ if there exists a polynomial-time uniform family $\{Q_x : x \in \Sigma^*\}$ of quantum circuits, where each Q_x has no input qubits and two output qubits, satisfying the following. For each $x \in \Sigma^*$, let ρ_x denote the two-qubit state output by Q_x and define Y_x and Z_x to be binary-valued random variables such that

$$\Pr\big((Y_x, Z_x) = (a, b)\big) = \langle a, b | \rho_x | a, b \rangle \tag{7.2}$$

for $a, b \in \{0, 1\}$. Then

$$x \in A_{\mathrm{yes}} \cup A_{\mathrm{no}} \implies \Pr(Y_x = 1) > 0, \tag{7.3}$$

$$x \in A_{\mathrm{yes}} \implies \Pr(Z_x = 1 \mid Y_x = 1) \geq \alpha(|x|), \tag{7.4}$$

$$x \in A_{\mathrm{no}} \implies \Pr(Z_x = 1 \mid Y_x = 1) \leq \beta(|x|). \tag{7.5}$$

The complexity class PostBQP is defined as $\mathrm{PostBQP}(2/3, 1/3)$.

This definition maps directly to the discussion at the start of the chapter: a standard basis measurement of the first output qubit of each Q_x produces the postselection bit, which is represented by the random variable Y_x, while a standard basis measurement of the second qubit produces the result bit, represented by the random variable Z_x. The condition (7.3) states that postselection must succeed with a nonzero probability, while (7.4) and (7.5) are error bounds on the result bit conditioned on postselection succeeding. When $\alpha = 2/3$ and $\beta = 1/3$, we obtain standard bounds for bounded-error computations.

Error reduction

Next we will prove that error can be reduced for PostBQP computations in much the same way as for BQP, through repetition followed by a threshold computation, naturally postselecting on all of the repetitions succeeding.

Theorem 7.2 (Error reduction for PostBQP). *Let $\alpha, \beta : \mathbb{N} \to [0,1]$ be polynomial-time computable functions for which there exists a polynomial resource bound q satisfying*

$$2^{-q(n)} \leq \beta(n) \leq \alpha(n) \leq 1 - 2^{-q(n)} \quad \text{and} \quad \alpha(n) - \beta(n) \geq \frac{1}{q(n)}, \tag{7.6}$$

for all $n \in \mathbb{N}$. It follows that $\mathrm{PostBQP}(\alpha, \beta) = \mathrm{PostBQP}$.

Proof. Once again, throughout the proof we will write $\alpha = \alpha(n)$, $q = q(n)$, and so on, taking the argument n as implicit for clarity.

We first prove that

$$\mathrm{PostBQP}(\alpha, \beta) \subseteq \mathrm{PostBQP}\big(1 - 2^{-r}, 2^{-r}\big) \tag{7.7}$$

for every polynomial resource bound r, which requires only the last of the inequalities in (7.6).

Suppose $A = (A_{\mathrm{yes}}, A_{\mathrm{no}}) \in \mathrm{PostBQP}(\alpha, \beta)$ and take $\{Q_x : x \in \Sigma^*\}$ to be a polynomial-time uniform family as in Definition 7.1. Let

$$N = 2rq^2 \qquad \text{and} \qquad t = \frac{\alpha + \beta}{2} \cdot N, \tag{7.8}$$

so that $N \geq 2r/(\alpha - \beta)^2$. Define a new family $\{R_x : x \in \Sigma^*\}$ in which each R_x runs N independent copies of Q_x on disjoint collections of qubits, producing outputs represented by pairs of random variables

$$(Y_x^1, Z_x^1), \ldots, (Y_x^N, Z_x^N) \tag{7.9}$$

when measured. As these are independent executions of Q_x, these pairs of random variables are independent, each distributed identically to (Y_x, Z_x) for the original circuit Q_x. The postselection qubit of R_x is then taken to be

$$Y_x^1 \wedge \cdots \wedge Y_x^N \tag{7.10}$$

while the result qubit is determined by a threshold computation — taking the value 1 if and only if at least t of the variables $Z_x^1, \ldots, Z_x^N$ take the value 1. This family is polynomial-time uniform: the value N and the threshold t are computable in

polynomial time because this is true of α and β, and the AND and threshold computations can be performed by circuits having size linear in N.

The postselection bit (7.10) of R_x takes the value 1 with probability $\Pr(Y_x = 1)^N$, which is positive whenever $x \in A_{\text{yes}} \cup A_{\text{no}}$. Conditioning on this event affects each pair separately, so the random variables $Z_x^1, \ldots, Z_x^N$ remain independent, with each taking the value 1 with probability

$$\lambda = \Pr(Z_x = 1 \mid Y_x = 1). \tag{7.11}$$

The probability that the result qubit of R_x takes the value 1, conditioned on its postselection qubit taking the value 1, is therefore

$$\sum_{\substack{y \in \{0,1\}^N \\ |y|_1 \geq t}} \lambda^{|y|_1} (1-\lambda)^{N-|y|_1}. \tag{7.12}$$

By Corollary 2.15, when $x \in A_{\text{yes}}$, we have $\lambda \geq \alpha$, and therefore (7.12) is at least $1 - 2^{-r}$; and when $x \in A_{\text{no}}$, we have $\lambda \leq \beta$, and therefore (7.12) is at most 2^{-r}. This establishes (7.7).

Taking $r = 3$ in (7.7) now gives

$$\text{PostBQP}(\alpha, \beta) \subseteq \text{PostBQP}\left(\tfrac{7}{8}, \tfrac{1}{8}\right) \subseteq \text{PostBQP}, \tag{7.13}$$

the second containment holding trivially, as $7/8 \geq 2/3$ and $1/8 \leq 1/3$.

For the reverse containment, note that (7.7) requires only that $\alpha - \beta$ is at least the reciprocal of a polynomial resource bound, so it holds equally when α and β are the constants $2/3$ and $1/3$. Taking $r = q$ in this case gives

$$\text{PostBQP} \subseteq \text{PostBQP}\left(1 - 2^{-q}, 2^{-q}\right) \subseteq \text{PostBQP}(\alpha, \beta), \tag{7.14}$$

the second containment holding trivially, as $\alpha \leq 1 - 2^{-q}$ and $\beta \geq 2^{-q}$. □

Closure of PostBQP under intersection

The following proposition will be used at the end of the chapter to prove the implication $A, B \in \text{PP} \implies A \cap B \in \text{PP}$ (for languages A and B) mentioned at the start of the chapter.

Proposition 7.3. *Let $A = (A_{\text{yes}}, A_{\text{no}})$ and $B = (B_{\text{yes}}, B_{\text{no}})$ be promise problems in* PostBQP *and define a promise problem $C = (C_{\text{yes}}, C_{\text{no}})$ as*

$$C_{\text{yes}} = A_{\text{yes}} \cap B_{\text{yes}}, \tag{7.15}$$

$$C_{\text{no}} = (A_{\text{yes}} \cap B_{\text{no}}) \cup (A_{\text{no}} \cap B_{\text{yes}}) \cup (A_{\text{no}} \cap B_{\text{no}}). \tag{7.16}$$

Then $C \in$ PostBQP.

Proof. Note that

$$C_{\text{yes}} \cup C_{\text{no}} = (A_{\text{yes}} \cup A_{\text{no}}) \cap (B_{\text{yes}} \cup B_{\text{no}}), \tag{7.17}$$

and therefore any string x satisfying the promise for C also satisfies the promise for both A and B. This fact will be used implicitly in the proof.

By Theorem 7.2 we have $A, B \in \text{PostBQP}(5/6, 1/6)$, so we may take

$$\{Q_x : x \in \Sigma^*\} \quad \text{and} \quad \{R_x : x \in \Sigma^*\} \tag{7.18}$$

to be polynomial-time uniform families satisfying Definition 7.1 for A and for B with $\alpha = 5/6$ and $\beta = 1/6$. Define a family $\{S_x : x \in \Sigma^*\}$ in which each S_x runs Q_x and R_x independently, taking its postselection and result bits to be the AND of the corresponding outputs of Q_x and R_x. This family is polynomial-time uniform.

Let us write (Y_x^A, Z_x^A) and (Y_x^B, Z_x^B) for the pairs of random variables associated with the circuits Q_x and R_x, as in Definition 7.1. The pairs (Y_x^A, Z_x^A) and (Y_x^B, Z_x^B) are independent, so the postselection bit of S_x therefore takes the value 1 with probability $\Pr(Y_x^A = 1)\Pr(Y_x^B = 1)$, which is positive. Moreover,

$$\begin{aligned} &\Pr\big(Z_x^A = 1 \wedge Z_x^B = 1 \;\big|\; Y_x^A = 1 \wedge Y_x^B = 1\big) \\ &\quad = \Pr(Z_x^A = 1 \mid Y_x^A = 1)\ \Pr(Z_x^B = 1 \mid Y_x^B = 1). \end{aligned} \tag{7.19}$$

If $x \in C_{\text{yes}}$, then both factors are at least $5/6$, so the product is at least $25/36 \geq 2/3$. If $x \in C_{\text{no}}$, then at least one factor is at most $1/6$ while the other is at most 1, so the product is at most $1/6 \leq 1/3$. Therefore $C \in \text{PostBQP}$. □

Remark 7.4. For languages A and B over an alphabet Σ, the proposition implies $A \cap B \in \text{PostBQP}$. In more detail, writing $\overline{A} = \Sigma^* \setminus A$ and $\overline{B} = \Sigma^* \setminus B$ for brevity, and viewing A and B as promise problems $(A, \overline{A})$ and $(B, \overline{B})$ as usual, this follows from

$$\big(A \cap \overline{B}\big) \cup \big(\overline{A} \cap B\big) \cup \big(\overline{A} \cap \overline{B}\big) = \Sigma^* \setminus (A \cap B). \tag{7.20}$$

7.2 PostBQP equals PP

Now we will prove that PostBQP = PP.

A lemma on approximations by powers of two

The following lemma will be needed for the proof that PP $\subseteq$ PostBQP, which is the more difficult of the two required containments.

Lemma 7.5. *Let $m \in \mathbb{N}$ and let $\gamma \in [1, 2^m]$. There exists $k \in \{0, \ldots, m\}$ such that*

$$\left|\gamma - 2^k\right| \leq \frac{2^k}{3}. \tag{7.21}$$

Proof. Let

$$I_j = \left(2^j - \tfrac{2^j}{3},\, 2^j + \tfrac{2^j}{3}\right] \tag{7.22}$$

for every $j \in \mathbb{Z}$. Because $2^j + 2^j/3 = 2^{j+1} - 2^{j+1}/3$ for each $j \in \mathbb{Z}$, these are disjoint intervals that partition the positive real numbers:

$$\bigcup_{j \in \mathbb{Z}} I_j = (0, \infty). \tag{7.23}$$

There must therefore exist a unique $k \in \mathbb{Z}$ such that $\gamma \in I_k$, which implies (7.21). It remains to prove that $k \in \{0, \ldots, m\}$. We have

$$2^{k-1} < \frac{2}{3} \cdot 2^k < \gamma \leq 2^m, \tag{7.24}$$

and therefore $k < m + 1$; as well as

$$2^{k+1} > \frac{4}{3} \cdot 2^k \geq \gamma \geq 1, \tag{7.25}$$

and therefore $k > -1$. As k is an integer, it follows that $k \in \{0, \ldots, m\}$. □

Main theorem and proof

Now we are prepared to prove the main result of the chapter, which states that PostBQP is equal to PP. The two required containments are proved in turn. The first, which is PostBQP $\subseteq$ PP, is straightforward using the GapP machinery of the previous chapter. The reverse containment, PP $\subseteq$ PostBQP, is the harder of the two; it is proved through a quantum algorithm for deciding if a GapP function takes a positive or negative value by making use of postselection. Lemma 7.5 is used in its analysis.

Theorem 7.6 (Aaronson)**.** PostBQP = PP.

Proof. First we will prove that PostBQP $\subseteq$ PP. Let $A = (A_{\text{yes}}, A_{\text{no}}) \in$ PostBQP and let $\{Q_x : x \in \Sigma^*\}$ be a polynomial-time uniform family of quantum circuits satisfying the conditions of Definition 7.1 for $\alpha = 2/3$ and $\beta = 1/3$. By Corollary 6.15, there exist GapP functions f and g along with a polynomial resource bound s such that

$$f(x, y, z) + i g(x, y, z) = 2^s \langle y | \rho_x | z \rangle \tag{7.26}$$

for all $y, z \in \{0,1\}^2$, where each ρ_x is the two-qubit state output by Q_x. In particular, given that density matrices have real diagonal entries, we have

$$f(x, ab, ab) = 2^s \langle a, b | \rho_x | a, b \rangle = 2^s \Pr(Y_x = a \wedge Z_x = b) \tag{7.27}$$

for every choice of $a, b \in \{0,1\}$, where Y_x and Z_x are the binary-valued random variables appearing in Definition 7.1.

By Proposition 6.6 it follows that

$$\begin{aligned} h(x) &= f(x, 11, 11) - f(x, 10, 10) \\ &= 2^s (\Pr(Y_x = 1 \wedge Z_x = 1) - \Pr(Y_x = 1 \wedge Z_x = 0)) \end{aligned} \tag{7.28}$$

is a GapP function. By Definition 7.1, $\Pr(Y_x = 1) > 0$ for every $x \in A_{\text{yes}} \cup A_{\text{no}}$. Thus, for every $x \in A_{\text{yes}}$ we have

$$\Pr(Z_x = 1 \mid Y_x = 1) \geq \frac{2}{3} > \frac{1}{3} \geq \Pr(Z_x = 0 \mid Y_x = 1) \tag{7.29}$$

and therefore

$$h(x) = 2^s \Pr(Y_x = 1) \big(\Pr(Z_x = 1 \mid Y_x = 1) - \Pr(Z_x = 0 \mid Y_x = 1) \big) > 0. \tag{7.30}$$

Along similar lines, for every $x \in A_{\text{no}}$ we have

$$\Pr(Z_x = 1 \mid Y_x = 1) \leq \frac{1}{3} < \frac{2}{3} \leq \Pr(Z_x = 0 \mid Y_x = 1) \tag{7.31}$$

and therefore

$$h(x) = 2^s \Pr(Y_x = 1) \big(\Pr(Z_x = 1 \mid Y_x = 1) - \Pr(Z_x = 0 \mid Y_x = 1) \big) < 0. \tag{7.32}$$

By Proposition 6.8 we conclude that $A \in \text{PP}$.

Now suppose $A = (A_{\text{yes}}, A_{\text{no}}) \in \text{PP}$. By Proposition 6.8 there exists a GapP function g such that $x \in A_{\text{yes}} \implies g(x) > 0$ and $x \in A_{\text{no}} \implies g(x) < 0$. Therefore, by Proposition 6.7, there exists a polynomial-time computable function $h : \Sigma^* \times \{0,1\}^* \to \{-1, 0, +1\}$ and a polynomial resource bound p such that

$$g(x) = \sum_{y \in \{0,1\}^p} h(x, y) \tag{7.33}$$

for all $x \in \Sigma^*$. In particular, for every $x \in A_{\text{yes}} \cup A_{\text{no}}$ we have $1 \leq |g(x)| \leq 2^p$.

Next, for every $x \in \Sigma^*$ and $k \in \{0, \ldots, p\}$, define a single-qubit pure state

$$|\psi_{x,k}\rangle = \frac{2^k |0\rangle + g(x) |1\rangle}{\sqrt{2^{2k} + g(x)^2}}. \tag{7.34}$$

These states can be prepared by postselection, as will be argued shortly. First, we will consider the distribution of measurement outcomes when these states are measured with respect to the $\{|+\rangle, |-\rangle\}$ basis. For reference, we have

$$|\langle +|\psi_{x,k}\rangle|^2 = \frac{(2^k + g(x))^2}{2^{2k+1} + 2g(x)^2}, \tag{7.35}$$

$$|\langle -|\psi_{x,k}\rangle|^2 = \frac{(2^k - g(x))^2}{2^{2k+1} + 2g(x)^2}. \tag{7.36}$$

In the case that $g(x) > 0$, we have

$$|\langle -|\psi_{x,k}\rangle|^2 = \frac{(2^k - g(x))^2}{2^{2k+1} + 2g(x)^2} = \frac{2^{2k} - 2g(x)2^k + g(x)^2}{2^{2k+1} + 2g(x)^2} < \frac{1}{2} \tag{7.37}$$

for every $k \in \{0, \ldots, p\}$. Moreover, as $g(x)$ is an integer and therefore at least 1, Lemma 7.5 implies there exists $k \in \{0, \ldots, p\}$ such that $|g(x) - 2^k| \leq 2^k/3$, and therefore

$$|\langle -|\psi_{x,k}\rangle|^2 = \frac{(2^k - g(x))^2}{2^{2k+1} + 2g(x)^2} \leq \frac{1}{9} \cdot \frac{2^{2k}}{2^{2k+1} + 2g(x)^2} < \frac{1}{18}. \tag{7.38}$$

That is, $|\langle +|\psi_{x,k}\rangle|^2 > 1/2$ for all $k \in \{0, \ldots, p\}$, and there exists $k \in \{0, \ldots, p\}$ such that $|\langle +|\psi_{x,k}\rangle|^2 > 17/18$.

The case that $g(x) < 0$ is symmetric: replacing $g(x)$ with $-g(x)$ in the analysis of the previous case provides the same bounds, with $|+\rangle$ and $|-\rangle$ exchanged. That is, $|\langle -|\psi_{x,k}\rangle|^2 > 1/2$ for all $k \in \{0, \ldots, p\}$, and there exists $k \in \{0, \ldots, p\}$ such that $|\langle -|\psi_{x,k}\rangle|^2 > 17/18$. In summary, for all $x \in A_{\text{yes}} \cup A_{\text{no}}$ and $k \in \{0, \ldots, p\}$ we have

$$\big|\langle \text{sgn}(g(x))|\psi_{x,k}\rangle\big|^2 > \frac{1}{2} \tag{7.39}$$

and there exists $k \in \{0, \ldots, p\}$ such that

$$\big|\langle \text{sgn}(g(x))|\psi_{x,k}\rangle\big|^2 > \frac{17}{18}. \tag{7.40}$$

Next let us address the creation of the states $|\psi_{x,k}\rangle$ through postselection, which can be done using the polynomial-time computable function $h : \Sigma^* \times \{0,1\}^* \to \{-1, 0, +1\}$ satisfying (7.33). To do this we will define two predicates,

$$R : \Sigma^* \times \mathbb{N} \times \{0,1\} \times \{0,1\}^* \to \{0,1\}, \tag{7.41}$$

$$S : \Sigma^* \times \{0,1\} \times \{0,1\}^* \to \{0,1\}, \tag{7.42}$$

as follows:

$$R(x,k,a,y) = [(a=0) \wedge ((y)_2 \geq 2^k)] \vee [(a=1) \wedge (h(x,y)=0)], \tag{7.43}$$

$$S(x,a,y) = [(a=1) \wedge (h(x,y)=-1)], \tag{7.44}$$

for all $x \in \Sigma^*$, $k \in \mathbb{N}$, $a \in \{0,1\}$, and $y \in \{0,1\}^*$. By the assumption that h is polynomial-time computable, the same is true of both R and S.

Now consider the state obtained by first preparing

$$\frac{1}{2^{(p+1)/2}} \sum_{a \in \{0,1\}} \sum_{y \in \{0,1\}^p} (-1)^{S(x,a,y)} |a\rangle |y\rangle |R(x,k,a,y)\rangle \tag{7.45}$$

in registers $(\mathsf{X},\mathsf{Y},\mathsf{Z})$, where X and Z are single qubits and Y is a p-qubit register; then performing a Hadamard gate on every qubit of Y; and finally postselecting on every qubit of (Y,Z) being in the 0 state. The creation of the states (7.45) is routine through standard methods (e.g., XORing $S(x,a,y)$ onto an ancillary qubit in the $|-\rangle$ state injects the phase through the phase kickback). Conditioned on this postselection, the state of the qubit X becomes the normalization of

$$\sum_{\substack{y \in \{0,1\}^p \\ (y)_2 < 2^k}} |0\rangle + \sum_{y \in \{0,1\}^p} h(x,y)|1\rangle = 2^k |0\rangle + g(x)|1\rangle, \tag{7.46}$$

which is $|\psi_{x,k}\rangle$.

Finally, consider the following procedure for finding the sign of $g(x)$, which uses postselection to create several copies of each state $|\psi_{x,k}\rangle$ for $k \in \{0,\ldots,p\}$. Specifically, it creates

$$N = 18(p+2) \tag{7.47}$$

copies of each of these states. (In fact, this is overkill. Taking $N = O(\log p)$ suffices, but we will avoid logarithms because we can.)

1. Prepare N copies of $|\psi_{x,k}\rangle$ for every $k \in \{0,\ldots,p\}$.
2. Measure every one of these states with respect to the basis $\{|+\rangle, |-\rangle\}$.
3. Accept if there exists a value $k \in \{0,\ldots,p\}$ for which at least $12(p+2)$ of the N measurement outcomes on the states $|\psi_{x,k}\rangle$ yield $+$, and reject otherwise.

To analyze the performance of this procedure, we make use of Corollary 2.15, with the parameters $\alpha = 5/6$, $\beta = 1/2$, and $r = p+2$. This is consistent with

$$N = 18(p+2) = \frac{2r}{(\alpha-\beta)^2} \tag{7.48}$$

measurements for each $k \in \{0, \ldots, p\}$, with the threshold value being

$$\frac{\alpha + \beta}{2} \cdot N = \frac{2}{3} \cdot N = 12(p+2). \tag{7.49}$$

Suppose first that $x \in A_{\text{yes}}$, so that $g(x) > 0$, and choose $k \in \{0, \ldots, p\}$ so that $|\langle + | \psi_{x,k} \rangle|^2 > 17/18 > \alpha$. (At least one such k exists, as argued above.) As the states prepared in step 1 are independent, so too are the measurement outcomes. Corollary 2.15 therefore implies that at least $12(p+2)$ of the N measurements associated with this value of k produce the outcome $+$ with probability at least $1 - 2^{-(p+2)}$, so the procedure accepts with probability at least

$$1 - 2^{-(p+2)} \geq \frac{3}{4}. \tag{7.50}$$

We need not consider any other values of k because they can only increase the probability of acceptance.

Now suppose that $x \in A_{\text{no}}$, so that $g(x) < 0$, and therefore

$$|\langle + | \psi_{x,k} \rangle|^2 < 1/2 = \beta \tag{7.51}$$

for every $k \in \{0, \ldots, p\}$. Again by the independence of the measurement outcomes, for each $k \in \{0, \ldots, p\}$, Corollary 2.15 implies that $12(p+2)$ or more of the N measurements of $|\psi_{x,k}\rangle$ produce the outcome $+$ with probability at most $2^{-(p+2)}$. By the union bound, the procedure therefore accepts with probability at most

$$(p+1)2^{-(p+2)} \leq \frac{1}{4}. \tag{7.52}$$

The procedure can be performed by a polynomial-time uniform family of quantum circuits, with postselection used to create the copies of the states $|\psi_{x,k}\rangle$. All of the required postselections can be combined into a single postselection qubit by computing NOR (i.e., the negation of the OR) of all of the qubits represented by the registers (Y, Z) described above, across all N copies of the states $|\psi_{x,0}\rangle, \ldots, |\psi_{x,p}\rangle$. Each individual postselection succeeds with nonzero probability, as $2^k|0\rangle + g(x)|1\rangle$ is a nonzero vector on account of $2^k \geq 1$, whatever the value of $g(x)$ may be, and therefore they all succeed with a nonzero (albeit very small) probability. This establishes (7.3).

Conditioned on this single postselection qubit taking the value 1, the copies prepared in the first step of the procedure are indeed in the states $|\psi_{x,k}\rangle$ analyzed above, so the bounds obtained in the two cases are bounds on the probability that the result qubit takes the value 1 conditioned on the postselection qubit taking the value 1. These are the conditions (7.4) and (7.5) for $\alpha = 2/3$ and $\beta = 1/3$, and it follows that $A = (A_{\text{yes}}, A_{\text{no}}) \in \text{PostBQP}$. □

Closure of PP under intersection

The following corollary follows immediately from Theorem 7.6 and Proposition 7.3.

Corollary 7.7. *Let* $A = (A_{\text{yes}}, A_{\text{no}})$ *and* $B = (B_{\text{yes}}, B_{\text{no}})$ *be promise problems in* PP *and define a promise problem* $C = (C_{\text{yes}}, C_{\text{no}})$ *as*

$$C_{\text{yes}} = A_{\text{yes}} \cap B_{\text{yes}}, \tag{7.53}$$

$$C_{\text{no}} = (A_{\text{yes}} \cap B_{\text{no}}) \cup (A_{\text{no}} \cap B_{\text{yes}}) \cup (A_{\text{no}} \cap B_{\text{no}}). \tag{7.54}$$

Then $C \in$ PP. *In particular, if* A *and* B *are languages in* PP, *then* $C = A \cap B \in$ PP.